VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belgaum-590018.



PROJECT WORK BIO-INSPIRED ALGORITHMS (23DS7BSBIO) REPORT

ON

SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms

Risk, return and tail-risk trade-offs with a live optimisation workbench

Submitted in Partial fulfilment for the VII Semester, B.E in Artificial Intelligence & Data Science

Submitted by:

Vivek Boora	**1BM23AD073**
Ayush Pandey	**1BM23AD074**
Nitin	**1BM24AD404**

Under the Guidance of:

Prof. Suman M
Assistant Professor
B.E, M.Tech, [PhD]
Dept. of AI & DS



2026-2027

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

B.M.S. College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

(Autonomous Institute under Visvesvaraya Technological University, Belagavi and Approved by AICTE, New Delhi, NAAC Accredited with 'A++' Grade)

B.M.S. College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

(Autonomous Institute under Visvesvaraya Technological University, Belagavi and Approved by AICTE, New Delhi, NAAC Accredited with 'A++' Grade)

Department of Artificial Intelligence & Data Science

CERTIFICATE

Certified that the BIO-INSPIRED ALGORITHMS [23DS7BSBIO] entitled "**SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms**" carried out by **Vivek Boora, Ayush Pandey and Nitin** bonafide students of B.M.S. College of Engineering, in partial fulfillment for the award of Bachelor of Engineering in Artificial Intelligence & Data Science of the Visvesvaraya Technological University, Belgaum during the year 2026-27. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library.

Signature of the Guide

Prof. Suman M
Asst. Prof. Dept. of AI & DS
BMSCE, Bengaluru.

Signature of the HOD

Dr. Seemanthini K
Assoc Prof. & Head, Dept. of AI & DS
BMSCE, Bengaluru.

Principal
Dr. Bheemsha Arya
B.M.S. College of Engineering

External Viva

	Name of the Examiners	Signature with Date
1.		
2.		

ABSTRACT

Dividing money across stocks involves conflicting goals: high return, low volatility and limited losses in bad periods. This project, SwarmFolio, treats long-only portfolio selection as a three-objective problem (annual return, annual volatility and Conditional Value-at-Risk) with a cap on each holding, a minimum holding size and an optional limit on the number of holdings. A multi-objective Particle Swarm Optimizer and an NSGA-II genetic algorithm are built on one shared repair operator, so that both search the same feasible space and can be compared fairly. The system is delivered as a Python library, a command-line tool and a browser workbench that animates both populations converging on their Pareto fronts and lets the user inspect any portfolio on them.

Six experiments on seeded simulated markets show that both algorithms find much better fronts than random search at an equal budget (hypervolume about 0.70 against 0.25) and never violate a constraint. They are close to each other: the swarm converges in far fewer iterations and gives a more even front, while the genetic algorithm ends marginally ahead from about 100 iterations. In 30 independent markets, portfolios chosen for the best Sharpe ratio fell from about 1.34 in training to about 0.53 on held-out data, no better than equal weight (0.59), whereas minimum-volatility portfolios had lower held-out volatility than equal weight in all 30 markets. A good front is therefore no promise of future return, and risk-oriented choices from the front are the more dependable.

DECLARATION

We the undersigned students of 7th semester, Department of Artificial Intelligence & Data Science, B.M.S. College of Engineering, declare that the project work for bio-inspired algorithms [23DS7BSBIO] entitled "**SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms**", is a bonafide work of us and our project is neither a copy nor by any means a modification of any other engineering project.

We also declare that this project was not entitled for submission to any other university in the past and shall remain the only submission made and will not be submitted by us to any other university in the future.

Name	**USN**	**Signature**
Vivek Boora	**1BM23AD073**	……
Ayush Pandey	**1BM23AD074**	……
Nitin	**1BM24AD404**	……

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our effort with success.

We are grateful to our institution, **B.M.S. College of Engineering**, with its ideals and inspirations for having provided us with the facilities, which has made this project a success.

We earnestly thank **Dr. Bheemsha Arya, Principal, BMS College of Engineering**, for facilitating academic excellence in the college and providing us with the congenial environment to work in, that helped us in completing this project.

We wish to extend our profound thanks to **Dr. Seemanthini K, Assoc Prof. and Head, Department of Artificial Intelligence & Data Science, BMSCE**, for giving us the consent to carry out this project.

We would like to express our sincere thanks to our guide **Prof. Suman M, Assistant Professor, Department of Artificial Intelligence & Data Science, BMSCE**, for the valuable guidance at every stage, which helped us in the successful completion of the project.

We would like to thank all the teaching and non-teaching staff for their valuable advice and support.

We would like to express our sincere thanks to our parents and friends for their support.

Vivek Boora

Ayush Pandey

Nitin

TABLE OF CONTENTS

TITLE	PAGE NO.
ABSTRACT	**i**
DECLARATION	**ii**
ACKNOWLEDGEMENTS	**iii**
LIST OF TABLES	**vi**
LIST OF FIGURES	**vii**

CHAPTER NO.	TITLE	PAGE NO.
1	**Introduction**	**1**
1.1	Overview	1
1.2	Motivation	1
1.3	Objectives	2
1.4	Scope	2
1.5	Sustainable Development Goals (SDGs)	3
1.6	Existing System	3
1.7	Proposed System	4
1.8	Work Plan	4
2	**Literature Survey**	**5**
2.1	Overview	5
2.2	Detailed Survey	5
2.2.1	Foundations of portfolio selection	5
2.2.2	Constrained and multi-objective formulations	5
2.2.3	Evolutionary and swarm algorithms	6
2.2.4	Assessing the quality of a Pareto front	6
2.3	Research Gap Identification	7
2.4	Mapping of your Objective with Research Gap Identification	8
3	**Requirement Analysis and Specification**	**9**
3.1	Functional Requirements	9
3.2	Non-functional Requirements	10
3.3	Hardware Requirements	10
3.4	Software Requirements	11
3.5	Cost Estimation	11
3.6	Assumptions	12
3.7	Risks and Challenges	12
3.7.1	AI Security Challenges	12
4	**Design**	**14**

CHAPTER NO.	TITLE	PAGE NO.
4.1	High Level Design	14
4.1.1	System Architecture	14
4.1.2	Abstract specification of Sub-systems	15
4.1.3	Interface Design	15
4.2	Methodology	18
4.2.1	Research Design	18
4.2.2	Data Collection	19
5	**Implementation**	**21**
5.1	Pseudo code	21
5.2	Use of appropriate advanced tools and techniques	24
5.3	Novelty	24
5.4	Mitigation of AI Security threats	25
6	**Testing and Validation**	**26**
6.1	Data interpretation	26
6.2	Analysis	26
6.3	Test results	27
6.3.1	Testing process	27
6.3.2	System test cases	27
7	**Results and Discussion**	**29**
7.1	Presentation of Results	29
7.1.1	Statistical Analysis	33
7.2	Figures	34
7.3	Subfigures	35
7.4	Discussion	38
7.5	Conclusion	40
8	**Conclusion**	**41**
8.1	Conclusion	41
8.2	Future Scope	42
	POs attained with Justification	43
	REFERENCES	44
	APPENDIX A: Details of list of publications related to this project	46
	APPENDIX B: POs and PSOs Mapped	47
	APPENDIX C: AI Generated Report and Plagiarism report Screen shot	50

LIST OF TABLES

Tables	Caption	Page No.
Table 1.1:	Relation of the project to the Sustainable Development Goals	3
Table 2.1:	Summary of the literature used in the project	7
Table 2.2:	Mapping of objectives to research gaps	8
Table 3.1:	Functional requirements	9
Table 3.2:	Non-functional requirements	10
Table 3.3:	Hardware requirements	10
Table 3.4:	Software requirements	11
Table 3.5:	Cost estimates	11
Table 3.6:	Risks and mitigations	12
Table 4.1:	Sub-systems and their responsibilities	15
Table 4.2:	HTTP interface of the server	17
Table 4.3:	Experimental design	18
Table 4.4:	Parameters of the synthetic market	20
Table 5.1:	Techniques and tools	24
Table 5.2:	Security threats and mitigations	25
Table 6.1:	System test cases and results	27
Table 7.1:	Front quality over 10 seeds (20 assets, 100 particles or individuals, 150 iterations)	29
Table 7.2:	Selected portfolios on the default market (seed 0): training and held-out performance	30
Table 7.3:	Effect of the holding limit (5 seeds each; 20 assets)	30
Table 7.4:	Scaling with the number of assets (3 seeds each for quality; run time measured separately with 3 seeds)	31
Table 7.5:	Hypervolume against budget (5 seeds each; one fixed reference for all rows)	31
Table 7.6:	Held-out performance over 30 independent markets (mean over markets; p is the paired Wilcoxon test of held-out Sharpe against equal weight)	32
Table 7.7:	Effect of removing an operator (8 seeds; limit of 8 holdings)	33
Table 7.8:	Summary of paired Wilcoxon signed-rank tests	33
Table 8.1:	Program Outcomes (PO) mapped to the project	48
Table 8.2:	Programme Specific Outcomes (PSO) mapped to the project	49

LIST OF FIGURES

Figures	Caption	Page No.
Figure 1.1:	Work plan of the project (weeks of the semester).	4
Figure 4.1:	Three-layer architecture of SwarmFolio.	14
Figure 4.2:	Main screen after a run: control rail on the left, risk-return stage on the right.	16
Figure 4.3:	The stage while the searches are running. Faint points are the current populations and solid points are the current non-dominated fronts.	16
Figure 4.4:	Portfolio inspector with selection controls, performance table and weights.	17
Figure 4.5:	Inline validation message for an infeasible constraint combination.	18
Figure 5.1:	Flowchart of the multi-objective particle swarm optimiser.	22
Figure 5.2:	Flowchart of the NSGA-II genetic algorithm.	23
Figure 5.3:	The repair operator shared by both algorithms.	23
Figure 7.1:	Hypervolume against the number of objective evaluations. Error bars are standard deviations over five seeds.	34
Figure 7.2:	Mean Sharpe ratio on the training and the held-out window for each strategy, over 30 markets.	34
Figure 7.3:	Hypervolume with and without the turbulence step (MOPSO) and the swap mutation (NSGA-II). Dots are individual seeds.	35
Figure 7.4:	Interface: held-out wealth curve of the selected portfolio against equal weight, with the comparison scorecard and the convergence chart.	35
Figure 7.5:	Fronts found on the training window of the default market: (a) return against volatility; (b) return against CVaR.	36
Figure 7.6:	Best training Sharpe on the front per iteration for seed 0 (a) and hypervolume distribution over ten seeds (b).	36
Figure 7.7:	Best-Sharpe portfolios on the default market: (a) weights of the largest holdings; (b) held-out wealth of selected portfolios against equal weight.	36
Figure 7.8:	Effect of the holding limit K: (a) hypervolume; (b) best training Sharpe on the front.	37
Figure 7.9:	Scaling with the number of assets: (a) measured run time; (b) hypervolume.	37
Figure 7.10:	Held-out distributions over 30 markets: (a) Sharpe ratio; (b) volatility; (c) maximum drawdown. Boxes show quartiles and whiskers the range excluding outliers.	37
Figure 7.11:	The same screen in the dark colour scheme.	38
Figure 7.12:	Phone-width layout: the stage comes first and the controls follow.	38

Chapter 1

Introduction

1.1 Overview

Portfolio selection is the problem of deciding how to divide a fixed amount of capital across a set of risky assets. The classical answer, mean-variance analysis, was formulated by Markowitz [1]: for every level of expected return there is a portfolio with the smallest possible variance, and the set of such portfolios forms the efficient frontier. An investor then chooses a point on this frontier that matches their appetite for risk.

Practical portfolio construction is harder than the textbook model for three reasons. First, investors care about more than one kind of risk: variance treats gains and losses symmetrically, whereas Conditional Value-at-Risk (CVaR) measures the average loss in the worst days [2]. Second, real mandates impose constraints such as a cap on any single holding, a minimum holding size and a limit on the number of assets held. A cardinality constraint turns the problem into a mixed-integer quadratic programme that exact methods solve slowly, which is why heuristics have been studied for it [3]. Third, the objectives conflict, so the honest output of an optimiser is a set of non-dominated trade-offs and not a single answer.

Population-based metaheuristics suit this setting. Particle Swarm Optimization (PSO) moves a swarm of candidate solutions through the search space using each particle's own best position and the best positions found by the swarm [4]. Genetic Algorithms (GA) evolve a population by selection, crossover and mutation [5], and the NSGA-II variant is the standard multi-objective form [6]. Both methods return a whole population in one run, which is exactly what is needed to approximate a Pareto front.

This project, named **SwarmFolio**, implements a multi-objective PSO (MOPSO) and an NSGA-II genetic algorithm on one shared portfolio model with three objectives (return, volatility and CVaR) and four constraints. The two optimisers are compared against random search and the equal-weight portfolio on training data and on held-out data. The work is packaged as a Python library with an automated test suite and as a browser-based workbench in which the user can watch both populations converge onto their fronts and inspect any portfolio on them.

1.2 Motivation

Three observations motivated this project. The first is that studies comparing PSO and GA for portfolio selection usually report results on one benchmark data set and in-sample only.

A front that looks excellent on the data it was fitted to may say little about future performance. Michaud [7] argued that mean-variance optimisers tend to amplify estimation error, and DeMiguel et al. [8] found that the naive 1/N rule was hard to beat out of sample across several optimised strategies. An evaluation that includes held-out data and many independent markets is therefore needed before any claim about usefulness can be made.

The second observation is that comparisons are often confounded by how constraints are handled. If each algorithm uses its own penalty or repair scheme, a difference in result may come from the constraint handling and not from the search strategy. Using one repair operator for both algorithms removes that source of difference.

The third is educational. Multi-objective optimisation is difficult to grasp from tables of numbers. Seeing a population of candidate portfolios move toward the efficient frontier, and then clicking a point to see its holdings and its behaviour on unseen data, makes the ideas of dominance, diversity and over-fitting concrete.

1.3 Objectives

1. To formulate long-only stock portfolio selection as a three-objective problem (maximise annual return, minimise annual volatility, minimise CVaR) with weight-cap, minimum-size and cardinality constraints.
2. To implement a multi-objective Particle Swarm Optimizer and an NSGA-II genetic algorithm that share one constraint-repair operator, so that they can be compared fairly.
3. To measure front quality with hypervolume, spacing, front size and runtime, and to compare both algorithms with random search at an equal evaluation budget.
4. To study how the algorithms respond to tighter constraints, larger asset universes, different search budgets and the removal of individual operators.
5. To test whether portfolios chosen from the fronts outperform the equal-weight portfolio on held-out data, using many independent simulated markets and a paired statistical test.
6. To build a browser-based interface that runs both optimisers, animates the search and lets the user inspect portfolios, with data upload for the user’s own prices.

1.4 Scope

In scope. Long-only equity portfolios; daily returns; historical (empirical) estimates of return, volatility and CVaR; the MOPSO and NSGA-II algorithms; random search and equal

weight as baselines; a synthetic market generator with fat tails and a crisis regime; CSV import of adjusted close prices; a Python library, command-line tool, local web server and test suite.

Out of scope. Live market data feeds and order execution; short selling and leverage; transaction cost as an optimisation objective (costs appear only in the optional walk-forward back-test); forecasting models for returns; and any claim that the portfolios are investment advice. The experimental study uses simulated markets, and real-market validation is listed as future work in Chapter 8.

1.5 Sustainable Development Goals (SDGs)

The project relates to the United Nations Sustainable Development Goals [9] in the following limited ways.

Table 1.1: Relation of the project to the Sustainable Development Goals

SDG	Relevance to this project
SDG 4: Quality Education	The workbench visualises dominance, Pareto fronts and over-fitting, and the code is open and documented, so it can be used to teach multi-objective optimisation.
SDG 8: Decent Work and Economic Growth	Transparent, constraint-aware portfolio tools support better-informed saving and investment decisions, and the held-out evaluation discourages over-confident claims.
SDG 9: Industry, Innovation and Infrastructure	The project applies bio-inspired computing to a financial decision problem and delivers it as reusable software with tests.

1.6 Existing System

Several approaches exist for the problem. Quadratic programming solvers return a single efficient portfolio for a chosen risk aversion and can trace the frontier by repeating the solve, but they do not handle cardinality directly. Single-objective heuristics, including PSO [10], [11], [12] and genetic algorithms [3], handle cardinality and other realistic constraints but typically optimise one scalarised objective, so a new run is needed for every trade-off. Multi-objective evolutionary algorithms return a front in one run [13], [14], and a review of this literature is given in [15].

What these systems usually lack is a common evaluation protocol. Constraint handling differs across papers, evaluation is mostly in-sample, and results are rarely accompanied by software that a student can run and inspect. Commercial portfolio tools are closed, which also limits their use in teaching.

1.7 Proposed System

SwarmFolio is an open, test-covered system with four parts. (i) A portfolio model with three objectives and a repair operator that maps any real vector to a feasible weight vector. (ii) Two optimisers, MOPSO and NSGA-II, that call the same model and therefore search the same feasible space. (iii) An evaluation layer with baselines, front-quality indicators, selection rules, held-out back-testing and an optional walk-forward test. (iv) A web workbench that runs jobs on a local server and animates the optimisers while they search.

The design choices that distinguish it from the existing approaches are the shared repair operator, the explicit three-objective formulation with CVaR, a hypervolume that is normalised across all methods compared in the same run, an out-of-sample study across 30 independent markets, and the interactive front inspector.

1.8 Work Plan

The work was organised into nine overlapping activities over a fifteen-week semester, as shown in the chart below. The chart is the plan that guided the work; activities overlapped because the interface and experiments depended on the optimisers being stable.

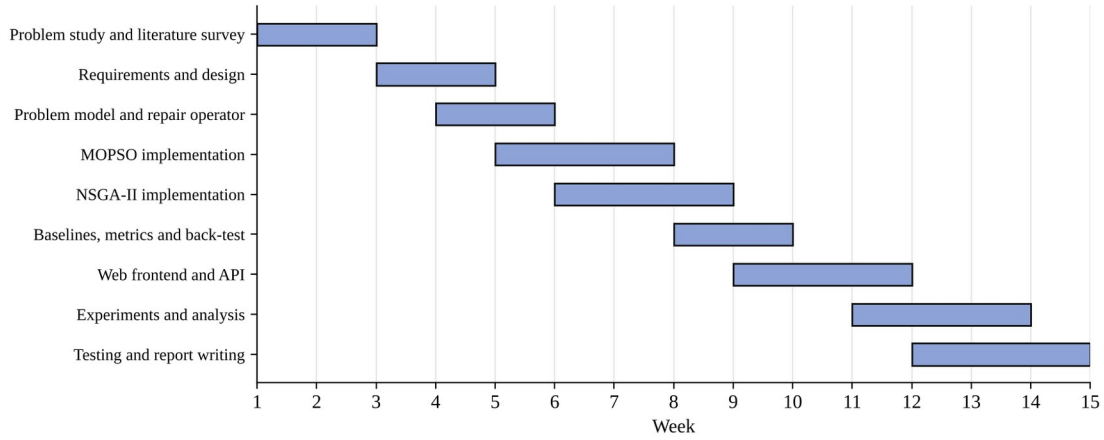


Figure 1.1: Work plan of the project (weeks of the semester).

Chapter 2

Literature Survey

2.1 Overview

The literature relevant to this project falls into four groups: the financial foundations of portfolio selection, the constrained and multi-objective formulations, the metaheuristics used to solve them, and the methods used to judge the quality of a Pareto front. This chapter summarises each group in the authors' own words and records the research gap that the project addresses.

2.2 Detailed Survey

2.2.1 Foundations of portfolio selection

Markowitz [1] introduced mean-variance portfolio selection and the efficient frontier. Sharpe [16] proposed the reward-to-variability ratio, now called the Sharpe ratio, which is used here to pick one portfolio from a front. Rockafellar and Uryasev [2] introduced the optimisation of Conditional Value-at-Risk, a tail-risk measure that is convex and can be minimised by linear programming. In SwarmFolio, CVaR is computed from the empirical distribution of daily portfolio returns and used as a third objective.

Two papers motivate the held-out evaluation. Michaud [7] argued that optimisers behave as error maximisers because they place large weights on assets whose historical means are over-estimated. DeMiguel, Garlappi and Uppal [8] compared many optimised strategies with the equal-weight rule and reported that none consistently beat it out of sample on the data they studied. Both results justify including the equal-weight portfolio as a baseline and judging claims on unseen data.

2.2.2 Constrained and multi-objective formulations

Chang et al. [3] studied the cardinality-constrained efficient frontier and applied genetic algorithms, tabu search and simulated annealing, showing that realistic constraints make the frontier discontinuous and hard to compute exactly. Anagnostopoulos and Mamanis [13] formulated portfolio selection as a tri-objective problem over risk, return and the number of securities and solved it with multi-objective evolutionary algorithms. Skolpadungket et al. [14] applied multi-objective genetic algorithms to portfolio optimisation with realistic constraints. Metaxiotis and Liagkouras [15] reviewed the application of multi-objective evolutionary algorithms to portfolio management and give a broader map of the field.

2.2.3 Evolutionary and swarm algorithms

Holland [5] established the genetic algorithm. Deb et al. [6] proposed NSGA-II, which combines fast non-dominated sorting, a crowding-distance measure that preserves diversity, and elitist survival of parents and offspring together. The simulated binary crossover (SBX) operator of Deb and Agrawal [17] gives real-coded GAs a crossover whose spread is controlled by a distribution index. Zitzler and Thiele [18] compared multi-objective evolutionary algorithms and introduced a size-of-dominated-space measure that is the origin of the hypervolume indicator.

Kennedy and Eberhart [4] introduced Particle Swarm Optimization, and Shi and Eberhart [19] added the inertia weight that balances exploration and exploitation. Coello Coello et al. [20] extended PSO to several objectives by keeping an external repository of non-dominated solutions that guides the swarm. For portfolio problems, Cura [10] applied PSO to the cardinality-constrained problem, Zhu et al. [11] to the constrained portfolio problem, and Deng et al. [12] proposed an improved PSO for Markowitz-based selection with cardinality constraints. Those three studies treat the problem with a single scalarised objective, whereas this project returns a three-objective front.

2.2.4 Assessing the quality of a Pareto front

Zitzler et al. [21] analysed performance indicators for multi-objective optimisers and showed that unary indicators can mislead unless their properties are understood. The hypervolume, which measures the region of objective space dominated by a front relative to a reference point, is compatible with Pareto dominance and is used as the primary indicator in this project. Schott [22] proposed the spacing metric, which measures how evenly the points of a front are distributed. Deb [23] gives the standard reference for these methods.

Table 2.1: Summary of the literature used in the project

Ref.	Focus	Method	Contribution used here
[1]	Portfolio theory	Mean-variance	Return-risk trade-off, efficient frontier
[16]	Performance measure	Reward-to-variability	Sharpe ratio for selecting a portfolio
[2]	Tail risk	CVaR optimisation	Third objective
[3]	Cardinality	GA, tabu search, annealing	Motivation for heuristics under constraints
[13]	Tri-objective model	MOEAs	Multi-objective formulation
[14]	Realistic constraints	Multi-objective GA	Constraint-aware GA precedent
[15]	Review	MOEAs for portfolios	Map of the field
[6]	Algorithm	NSGA-II	Basis of the GA
[20]	Algorithm	Multi-objective PSO	Basis of the swarm optimiser
[10], [11], [12]	Portfolio PSO	Single-objective PSO variants	Evidence that PSO suits the constrained problem
[7], [8]	Out-of-sample evidence	Empirical comparison	Motivates held-out tests and the 1/N baseline
[21], [22]	Indicators	Hypervolume, spacing	Evaluation of fronts

2.3 Research Gap Identification

From the literature reviewed in this chapter, four gaps are visible.

- **Gap 1: evaluation protocol.** The PSO and GA portfolio studies reviewed report results on benchmark data and mainly in-sample. Held-out results across many independent markets, with a paired test against the equal-weight rule, were not found in the papers surveyed here.
- **Gap 2: confounded comparison.** Constraint handling differs between algorithms in the surveyed studies, so a difference in outcome cannot be attributed to the search strategy alone.
- **Gap 3: tail risk with swarms.** The PSO studies reviewed here [10], [11], [12] optimise a single scalarised objective, so a three-objective front including CVaR is not produced in those works.
- **Gap 4: inspectability.** Few of the surveyed works ship software in which the search can be observed and a candidate portfolio inspected, which limits their value as teaching and exploration tools.

These gaps describe the papers surveyed for this report, which are not an exhaustive review of the field. The claim is therefore limited to the surveyed literature.

2.4 Mapping of your Objective with Research Gap Identification

Table 2.2: Mapping of objectives to research gaps

Objective (Section 1.3)	Gap addressed	How the project addresses it
1. Three-objective model with constraints	Gap 3	Return, volatility and CVaR are optimised together under cap, floor and cardinality constraints.
2. Shared repair operator, MOPSO and NSGA-II	Gap 2	Both algorithms call the same repair operator and objective function, so the feasible space is identical.
3. Front-quality indicators and baselines	Gap 1	Hypervolume (shared normalisation), spacing, front size, runtime, random search at equal budget.
4. Constraint, scale, budget and ablation studies	Gaps 1 and 2	Experiments E2, E3, E4 and E6 (Section 4.2.1) vary one factor at a time.
5. Held-out test over many markets	Gap 1	30 independent simulated markets, paired Wilcoxon test against equal weight (Chapter 7).
6. Browser workbench	Gap 4	Live animation of both populations and a click-to-inspect portfolio panel.

Chapter 3
Requirement Analysis and Specification

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	The system shall accept a market either from a built-in synthetic generator (assets, trading days, seed) or from an uploaded CSV of prices.
FR2	The system shall let the user set the maximum weight per asset, the minimum holding size and an optional limit on the number of holdings, and shall reject combinations that cannot be satisfied.
FR3	The system shall let the user set the population size, the number of generations and the share of history used for training.
FR4	The system shall run MOPSO and NSGA-II on the same problem and return a non-dominated set of feasible portfolios for each.
FR5	Every returned portfolio shall be long-only, fully invested and within the weight cap and cardinality limit.
FR6	The system shall show the progress of both searches while they run, including the current population and front.
FR7	The system shall let the user stop a running job.
FR8	The system shall compute hypervolume, spacing, front size, best Sharpe ratio and runtime for each method, with random search as a baseline.
FR9	The system shall let the user select a portfolio from a front by clicking, by a slider or by a rule (best Sharpe, lowest volatility, balanced) and show its weights.
FR10	For a selected portfolio the system shall show training and held-out return, volatility, Sharpe ratio and drawdown next to the equal-weight portfolio, and a held-out wealth curve.
FR11	The system shall report invalid input and server errors in the interface in plain language.
FR12	The library shall also be usable from Python and from a command-line tool that writes tables and figures.

3.2 Non-functional Requirements

Table 3.2: Non-functional requirements

Category	Requirement
Correctness	Constraint satisfaction shall hold for every portfolio produced; objective values shall match an independent calculation. Both are covered by automated tests.
Reproducibility	A run with the same data, parameters and seed shall give the same front.
Performance	A default run (100 particles or individuals, 150 iterations, 20 assets, 1,500 days) shall finish within about ten seconds on a modern laptop, and the interface shall stay responsive during a run.
Usability	Controls shall be labelled in plain language, work with a keyboard, and the layout shall work from phone width to desktop width in light and dark colour schemes.
Portability	The server shall use only the Python standard library; the interface shall need no build step and no network access.
Security	Inputs shall be validated and bounded; static files shall be served only from the web directory; user-supplied text shall be escaped before display. See Section 3.7.1.
Maintainability	Modules shall have single responsibilities, and behaviour shall be covered by tests that run in under ten seconds.

3.3 Hardware Requirements

The project is software only. The development and experiments were run in a cloud container with four virtual CPU cores. The minimum recommended configuration to use the system is listed below.

Table 3.3: Hardware requirements

Component	Quantity	Specification
Computer (laptop or desktop)	1	Dual-core 64-bit CPU or better, 4 GB RAM, 500 MB free disk
Display	1	Any display of phone size or larger
Network	-	Not required to run; needed only to install the Python packages

3.4 Software Requirements

Table 3.4: Software requirements

Software	Version used	Purpose
Python	3.13	Language of the library, server and experiments
NumPy [24]	2.5	Vectorised portfolio evaluation and the optimisers
pandas [25]	3.0	Time-indexed returns, CSV import
Matplotlib [26]	3.11	Plots written by the command-line tool and the report figures
SciPy [27]	1.18	Wilcoxon signed-rank test in the report analysis only
pytest	-	Automated tests
Web browser	Any current version	HTML, CSS and JavaScript interface (canvas)
Chromium with Playwright	-	Automated interface checks and the screenshots in this report

3.5 Cost Estimation

All software used is free and open source, so the incremental cost of the project is small. The table gives an estimate; the figures for the computer represent equipment that the team already owns and are shown only to indicate what is needed.

Table 3.5: Cost estimates

Item	Estimated cost (INR)	Note
Python, NumPy, pandas, Matplotlib, SciPy, pytest	0	Open-source licences
Web interface (HTML, CSS, JavaScript)	0	No framework or build tool
Cloud container for development and experiments	0	Provided with the development environment used
Printing and binding of the report	to be filled by the team	Depends on the department’s requirements
Computer for running the system	0 (existing)	Not purchased for the project

3.6 Assumptions

- Daily returns are the unit of analysis, and 252 trading days make a year.
- The historical empirical distribution of returns on the training window is an adequate basis for estimating return, volatility and CVaR. This is the assumption that the held-out tests probe.
- Positions are long-only and fully invested, with no leverage, no cash and no short selling.
- Trading costs and taxes are ignored in the optimisation. A flat cost on turnover is applied only in the optional walk-forward back-test.
- The simulated markets are a controlled testbed and not a model of any particular exchange.

3.7 Risks and Challenges

Table 3.6: Risks and mitigations

Risk	Effect	Mitigation
Over-fitting to the training window	Fronts look strong in-sample and weaken on unseen data	Held-out evaluation, equal-weight baseline, 30 independent markets, honest reporting
Constraint infeasibility	Impossible configuration, for example a 10% cap with five holdings	Validation in the interface and in the problem constructor with a clear error
Premature convergence	Population collapses onto few holdings	PSO turbulence and GA swap mutation, with ablation tests (Section 7.4)
Metric misuse	Hypervolume depends on the reference point	One shared normalisation for all methods compared in a run
Long runtime at large scale	Slow response in the interface	Bounded parameters, vectorised evaluation, progress display and a Stop button
Synthetic data not representative	Conclusions may not transfer to real markets	Stated as a limitation; CSV import supports real data for future validation

3.7.1 AI Security Challenges

Although the system contains no learned model, it is an AI system in the sense that it accepts user data and returns decision support, so several security concerns apply. They are listed here and their handling is described in Section 5.4.

- **Malicious or malformed input.** An uploaded CSV could be malformed, oversized or crafted to exhaust memory or time.
- **Injection into the interface.** Column names in an uploaded file are displayed as asset names and could carry markup intended to run script in the browser.
- **Resource exhaustion.** Unbounded population sizes, iteration counts or concurrent jobs could stall the server.
- **Path traversal.** A static file handler could be tricked into serving files outside the web directory.
- **Data manipulation.** A user (or an upstream data error) can bias the optimiser by supplying outlier prices, which the optimiser will exploit.
- **Unauthorised access.** The server has no authentication and is intended for local use.

Chapter 4

Design

4.1 High Level Design

4.1.1 System Architecture

SwarmFolio has three layers, shown in the figure below. The browser presents controls and charts and exchanges JSON with a small server. The server owns the jobs: it starts each optimisation on worker threads, stores the most recent snapshot of each population, and returns it when the browser polls. The optimisation core is an ordinary Python package that does not depend on the server, so the same code is used by the command-line tool, the tests and the web application.

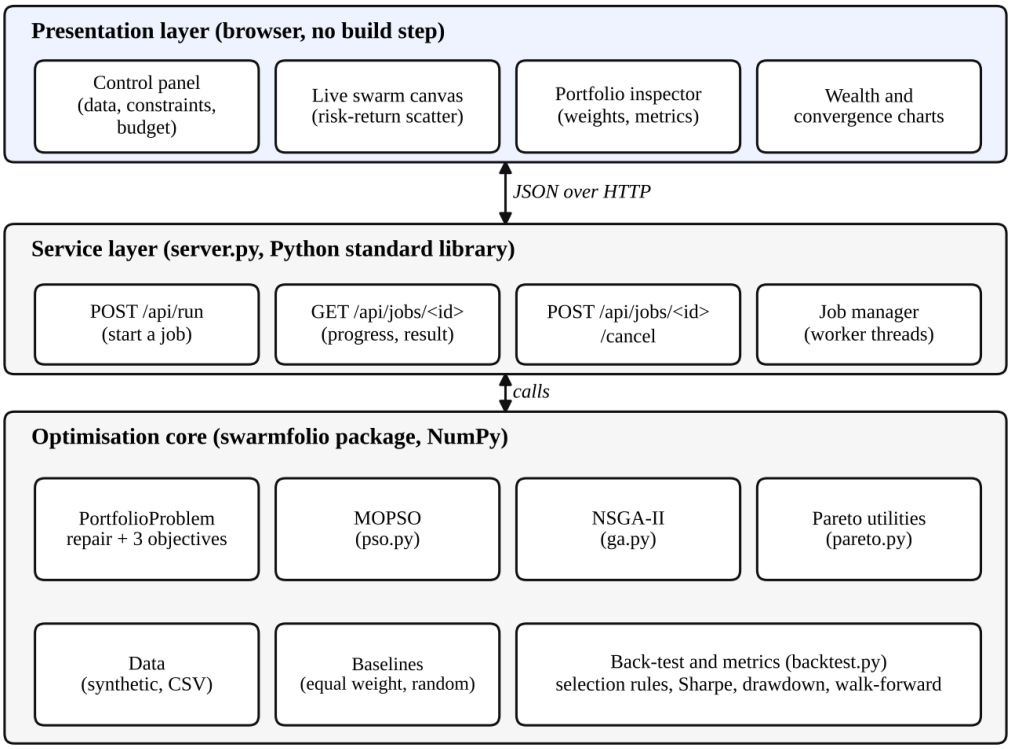


Figure 4.1: Three-layer architecture of SwarmFolio.

A run proceeds as follows. The browser sends the configuration to the run endpoint. The server validates it, builds the market and the portfolio problem, draws 300 random feasible portfolios as a visual backdrop, and starts MOPSO and NSGA-II on separate threads. Each optimiser calls a callback once per iteration, which records the current population and front. The browser polls the job about nine times a second and redraws. When both optimisers finish, the server runs the random-search baseline, scores every front on the held-out data, selects portfolios by the three rules, and publishes the final result.

4.1.2 Abstract specification of Sub-systems

Table 4.1: Sub-systems and their responsibilities

Sub-system (module)	Responsibility	Main inputs and outputs
Data (data.py)	Generate the synthetic market or load prices from CSV	Seed, size, or a file; returns a table of daily returns
Problem (problem.py)	Define objectives and constraints; repair; evaluate	Return matrix and constraint settings; weights in, objective rows out
MOPSO (pso.py)	Multi-objective swarm search with an external archive	Problem and swarm parameters; non-dominated set and history
NSGA-II (ga.py)	Multi-objective genetic search	Problem and GA parameters; non-dominated set and history
Pareto utilities (pareto.py)	Non-dominated sorting, crowding distance, hypervolume, spacing	Objective matrices
Baselines (baselines.py)	Equal-weight portfolio and random search	Problem and a budget
Back-test (backtest.py)	Selection rules, performance metrics, walk-forward test	Fronts and held-out returns
Server (server.py)	Job manager, JSON API, static files	HTTP requests; JSON responses
Web interface (web/)	Controls, live canvas, inspector, charts	User actions; API responses
Command line (cli.py)	Batch comparison that writes tables and plots	Arguments; files

4.1.3 Interface Design

The interface is a single page with a control rail on the left and a working area on the right. On narrow screens the working area comes first and the rail follows. The design aims to keep the swarm visualisation as the focus and to keep everything else quiet. Colour is used for data only: blue stands for the particle swarm, orange for the genetic algorithm and grey for random portfolios, and the same colours are used in every chart. A light and a dark theme are provided and follow the system setting.

The stage shows annual return against volatility, with an option to switch the horizontal axis to CVaR. Dots for each algorithm are coloured, and the legend entries can be toggled. Two thin meters show how far each search has progressed. After the run, the chart shows the two fronts as lines with points, an equal-weight marker, and a ring on the selected portfolio.

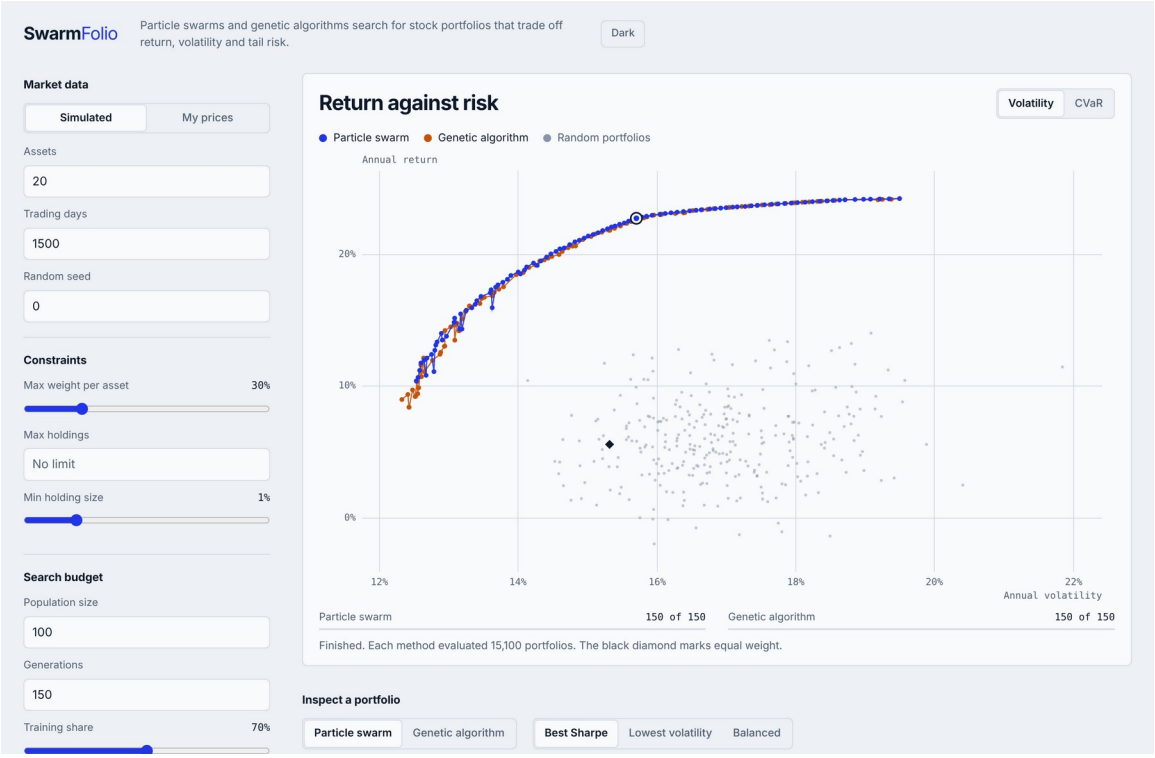


Figure 4.2: Main screen after a run: control rail on the left, risk-return stage on the right.

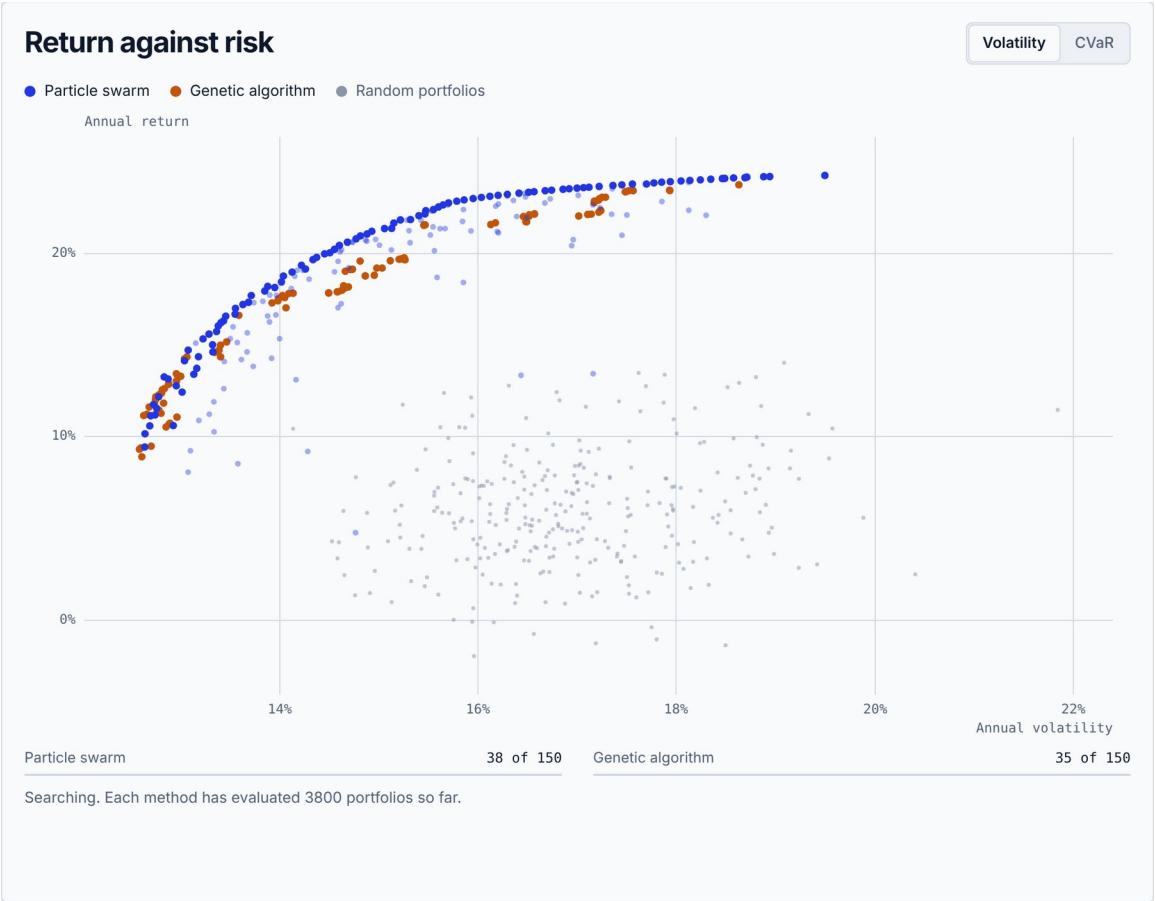


Figure 4.3: The stage while the searches are running. Faint points are the current populations and solid points are the current non-dominated fronts.

Below the stage, the inspector lets the user choose an algorithm and a selection rule, or drag a slider along the front. It shows a table of training and held-out performance beside the

equal-weight portfolio, the weights as horizontal bars, and the held-out wealth curve. A scorecard and a convergence chart summarise the comparison of the methods.



Figure 4.4: Portfolio inspector with selection controls, performance table and weights.

The browser and server communicate through three endpoints, summarised below.

Table 4.2: HTTP interface of the server

Endpoint	Method	Purpose
/api/run	POST	Start a job. Body: JSON configuration (data source, constraints, budget). Returns the job id. Rejects invalid values with a message.
/api/jobs/<id>	GET	Poll a job. Returns status, per-algorithm progress, the latest population and front of each algorithm and, when done, the full result.
/api/jobs/<id>/cancel	POST	Stop a running job.

Errors are shown where the user can act on them. For example, if the cap and holding limit cannot both be met, the message appears under the constraint fields and the run button is disabled, as in the figure below.

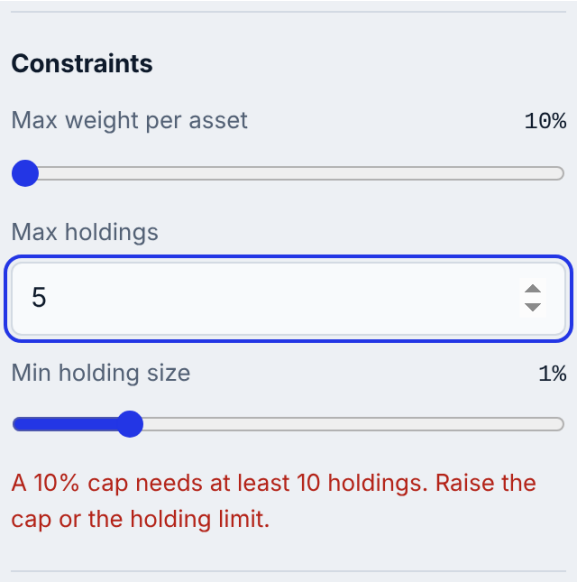


Figure 4.5: Inline validation message for an infeasible constraint combination.

4.2 Methodology

4.2.1 Research Design

The study is a controlled computational experiment. All algorithms see the same training data and the same constraints, and each is given the same number of objective evaluations. Six experiments were run. Each used 5 to 30 independent seeds so that variation could be reported and tested.

Table 4.3: Experimental design

ID	Question	Design
E1	Does each algorithm approximate the front better than random search, and do the algorithms differ?	Default setting (20 assets, 1,500 days), 10 seeds, hypervolume with shared normalisation, spacing, runtime
E2	How do tighter cardinality limits affect front quality?	Limit K of 20 (none), 10, 8 and 5 holdings, 5 seeds each
E3	How do results and runtime scale with the number of assets?	10, 20, 40 and 60 assets, 3 seeds each
E4	How fast does front quality improve with search budget?	5, 10, 25, 50, 100 and 200 iterations, 5 seeds each
E5	Do portfolios chosen from the fronts outperform equal weight on unseen data?	30 independent simulated markets; PSO, GA and random search each pick best-Sharpe and minimum-volatility portfolios on the training window; evaluated on held-out data
E6	Do the PSO turbulence and GA swap-mutation operators help?	Full versus operator removed, cardinality limit 8, 8 seeds

Indicators. Hypervolume is the fraction of a normalised objective box that is dominated by a front. The box runs from the best value of each objective over all fronts being compared to

a reference point just beyond the worst value, so that methods compared in the same run share one scale. It is estimated by Monte Carlo sampling with 200,000 points. Spacing is the Schott metric [22]. Front size is the number of non-dominated points. Runtime is wall-clock time of one run.

Held-out metrics. For a daily return series r_t of a portfolio the annualised return is 252 times the mean, the annualised volatility is the sample standard deviation times the square root of 252, and the Sharpe ratio is their quotient (risk-free rate taken as zero). Maximum drawdown is the largest peak-to-trough fall of cumulative wealth.

Statistical test. For E5 the held-out Sharpe ratio of each strategy is compared with that of equal weight over the 30 markets using the paired Wilcoxon signed-rank test [28] implemented in SciPy [27]. For E1 the hypervolumes of two algorithms over the seeds are compared with the same paired test.

4.2.2 Data Collection

No licensed market data was used. The experiments use a seeded synthetic market, which makes every result reproducible and lets the experiments be repeated on many independent markets. Returns of asset i on day t are generated from a one-factor market model plus sector factors and idiosyncratic noise:

$$r_{i,t} = \alpha_i + \beta_i \, m_t + s_{k(i),t} + \sigma_i \, \varepsilon_{i,t} \qquad (1)$$

where m_t is the market factor, $s_{k(i),t}$ the factor of the asset's sector, and ε a standardised Student-t variable with five degrees of freedom, which gives fat tails. The parameters are listed in the table below. A crisis regime between 45% and 51% of the sample multiplies returns by 2.2 and adds a negative drift proportional to beta, so that volatility and tail risk do not move together with mean return.

Table 4.4: Parameters of the synthetic market

Component	Setting
Market factor m_t	Mean 7% per year; daily volatility 0.9%; Student-t, 5 d.o.f.
Sector factors	4 sectors; daily volatility 0.5%; Student-t, 5 d.o.f.
Asset beta β_i	Uniform on [0.5, 1.4]
Asset alpha α_i	Uniform on [-2%, 10%] per year
Idiosyncratic volatility σ_i	Uniform on [0.6%, 2.0%] per day
Crisis regime	Days 45% to 51% of the sample: returns x 2.2, drift -0.15% x beta per day
Default size	20 assets, 1,500 business days from 1 January 2018
Train / test split	70% / 30% in time order (first 1,050 days train, last 450 days test)

The CSV loader accepts a wide file with a date column and one adjusted-close column per ticker. It drops tickers with more than 5% missing prices, forward-fills gaps, and converts prices to daily simple returns. The same train and test split is applied.

Chapter 5

Implementation

5.1 Pseudo code

For a portfolio w in R^N (N assets) and a matrix R of T daily returns, the three objectives that are minimised are

$$f_1(w) = -252 \cdot \text{mean}(Rw) \tag{2}$$

$$f_2(w) = \sqrt{252} \cdot \text{std}(Rw) \tag{3}$$

$$f_3(w) = -(\text{mean of the k lowest values of } Rw) \tag{4}$$

where k is the ceiling of 0.05 T. All three are computed for a whole population at once as a matrix product followed by a sort, which is why a population of 100 can be evaluated in well under a millisecond.

Algorithm 1: Constraint repair

```
function REPAIR(x):                     # any real vector -> feasible weights
    w <- max(x, 0)
    if K is set: keep the K largest entries of w, zero the rest
    normalise w to sum 1; set entries below w_min to 0
    if fewer than ceil(1/w_max) entries are non-zero:
        activate further assets in order of x with a small weight
    normalise w to sum 1
    repeat while some w_i > w_max:
        excess <- sum over i of (w_i - w_max); set those w_i to w_max
        give the excess to the other non-zero assets in proportion to w
        (if all are capped, give it to the next assets in order of x)
    return w / sum(w)
```

Algorithm 2: Multi-objective particle swarm optimiser

```
function MOPSO(problem, N, T, archive_size):
    X <- REPAIR(random vectors); V <- small random velocities
    F <- EVALUATE(X); PBEST <- X; ARCHIVE <- non-dominated(X, F)
    for t = 1..T:
        w <- 0.9 - 0.5 * t/T                        # linearly decreasing inertia
        for each particle i:
            leader_i <- binary tournament on ARCHIVE by crowding distance
            V_i <- w*V_i + 1.5*r1*(PBEST_i - X_i) + 1.5*r2*(leader_i - X_i)
            clip V_i to [-0.3, 0.3]; X_i <- X_i + V_i
            with probability 0.1: add Gaussian noise (sd 0.1) to 30% of the entries
        X <- REPAIR(X); F <- EVALUATE(X)
        for each i: if F_i dominates PBEST_i, or neither dominates (50% chance):
            PBEST_i <- X_i
        ARCHIVE <- non-dominated(ARCHIVE + X); remove duplicates
        while |ARCHIVE| > archive_size: remove the most crowded point
    return ARCHIVE
```

Algorithm 3: NSGA-II genetic algorithm

```
function NSGA2(problem, P, G):
    X <- REPAIR(random vectors); F <- EVALUATE(X)
    for g = 1..G:
        rank, crowd <- NON_DOMINATED_SORT(F), CROWDING_DISTANCE(F)
        parents <- binary tournament (lower rank wins; ties: larger crowding)
        children <- SBX crossover(parents, eta_c=15, p_c=0.9)
        children <- polynomial mutation(eta_m=20); swap mutation with p=0.2
                    (move the weight of one held asset to a random asset)
        children <- REPAIR(children); F_c <- EVALUATE(children)
        pool <- X + children; sort pool by (rank, -crowding)
        X, F <- first P members of the sorted pool
    return rank-0 members of X
```

The two flowcharts below show the same procedures graphically, and the third figure shows the repair operator that both algorithms share.

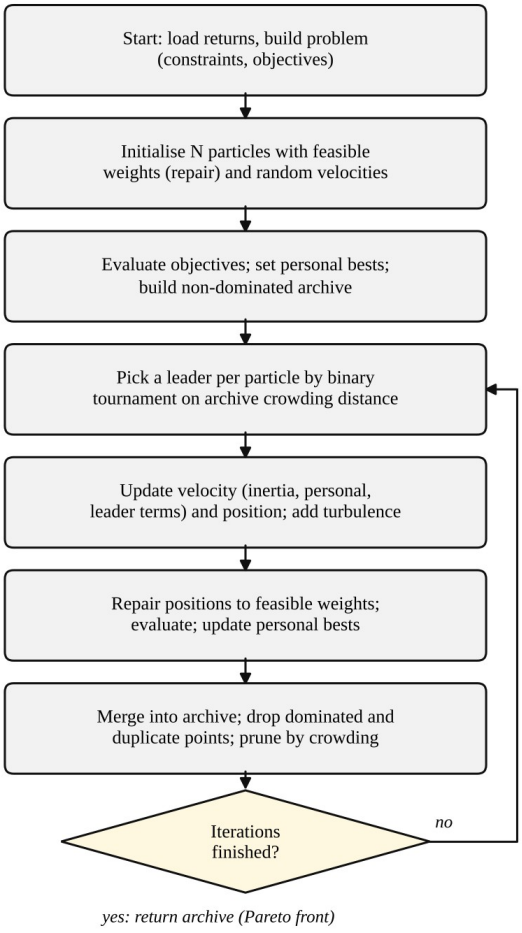


Figure 5.1: Flowchart of the multi-objective particle swarm optimiser.

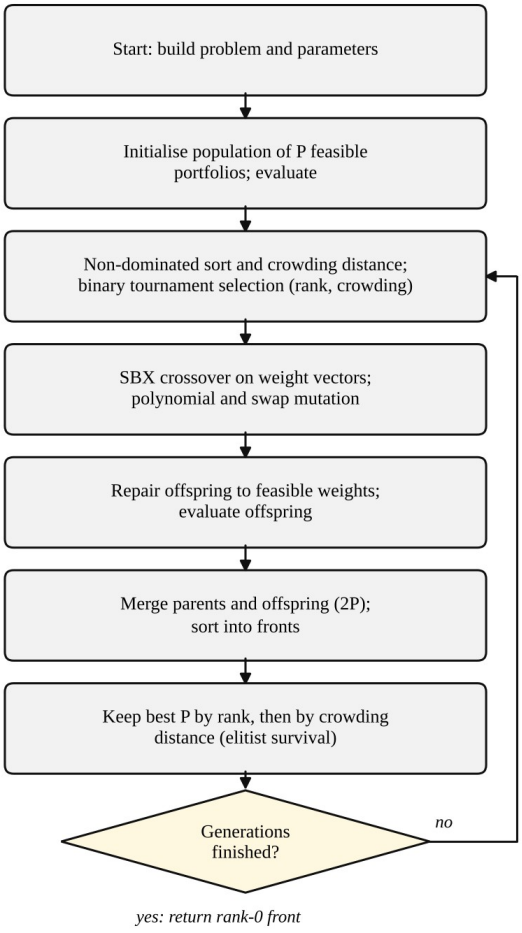


Figure 5.2: Flowchart of the NSGA-II genetic algorithm.



Figure 5.3: The repair operator shared by both algorithms.

5.2 Use of appropriate advanced tools and techniques

Table 5.1: Techniques and tools

Technique or tool	Where it is used	Reason
Fast non-dominated sorting [6]	NSGA-II survival and tournament	Orders solutions by Pareto rank in one pass over a dominance matrix
Crowding distance [6]	NSGA-II, MOPSO leader choice and archive pruning	Keeps the front spread out
External archive [20]	MOPSO	Stores non-dominated solutions that guide the swarm
Inertia weight schedule [19]	MOPSO	Explores early and refines late
Simulated binary crossover [17]	NSGA-II	Real-coded crossover with controlled spread
Vectorised evaluation (NumPy [24])	All objective calculations	One matrix product evaluates the whole population
Monte Carlo hypervolume	Front comparison	A three-objective hypervolume without a specialised library
Seeded generators	Everything stochastic	Exact reproducibility of every experiment
Worker threads with progress callbacks	Server	Both searches run side by side and can be observed and cancelled
Canvas rendering with eased axes	Interface	Smooth animation of hundreds of points without a charting library

5.3 Novelty

The project does not propose a new optimisation algorithm; both optimisers are established methods. Its contribution lies in how they are set up, compared and presented:

- **One feasible space for both optimisers.** A single repair operator is applied to every position, offspring and initial candidate, so differences between the algorithms are not confounded by constraint handling.
- **Three-objective formulation with CVaR** solved by a swarm and a GA in the same framework, with cardinality, cap and floor constraints.
- **Shared-normalisation hypervolume.** Methods compared in one run use the same ideal and reference points, so their hypervolumes are directly comparable.
- **Operator ablations.** The PSO turbulence step and the GA swap mutation, which are specific to this implementation, are tested by removing them (E6).

- **Evaluation on many markets.** The held-out comparison with equal weight uses 30 independent simulated markets and a paired test, rather than one data set.
- **Observable search.** The interface animates both populations during the run and lets the user inspect any portfolio on the final fronts.

5.4 Mitigation of AI Security threats

Table 5.2: Security threats and mitigations

Threat (Section 3.7.1)	Mitigation implemented	Verified by
Malformed or oversized input	Request bodies are limited to 8 MB; the JSON body must be an object; numeric parameters are range-checked (4 to 60 assets, 300 to 5,000 days, 10 to 300 population, 5 to 500 iterations); CSV parse errors are returned as messages	Server tests for out-of-range input; manual upload test
Script injection through asset names	All text derived from uploaded files is escaped before it is placed in the page	Code review of the rendering functions
Resource exhaustion	Parameter bounds; a new run cancels earlier jobs; only the eight most recent jobs are kept; progress and cancel endpoints	Stop-button test
Path traversal	Requested paths are resolved and must lie inside the web directory; otherwise the server returns 404	Server test requesting a parent-directory path
Data manipulation or outliers	Held-out evaluation and an equal-weight baseline expose over-fitting; the weight cap and cardinality limit reduce concentration; no code is executed from data	Experiments E5; test suite
Unauthorised access	The server binds to the local machine by default and has no accounts; the report states that it is for local use and must not be exposed to the internet without authentication	Default bind address

These measures are proportionate to a locally run teaching and research tool. They do not make the server suitable for public deployment, and a hosted version would need authentication, rate limiting and an isolated worker process.

Chapter 6

Testing and Validation

6.1 Data interpretation

The default market has 20 assets and 1,500 business days. In time order, the first 1,050 days form the training window on which every optimiser works, and the last 450 days are held out and used only to evaluate the chosen portfolios. On the training window the annualised mean returns of the single assets range from -13.2% to 35.9% and their annualised volatilities from 16.5% to 41.2%. A portfolio of these assets can reach a much lower volatility than any single asset, which is the diversification effect the optimisers exploit.

The three objectives are interpreted as follows. Return is the mean daily return multiplied by 252. Volatility is the standard deviation of daily returns multiplied by the square root of 252. CVaR is the average loss on the worst 5% of days, expressed as a daily percentage, so a CVaR of 1.9% means that on its worst days the portfolio lost on average 1.9%. A point on the front is better than another if it is no worse on all three objectives and better on at least one. Because return is maximised while the other two are minimised, the front rises to the right in the return-volatility plane: more return can only be had by accepting more risk.

Two cautions apply to reading the results. First, the training numbers are in-sample: they describe how well a portfolio would have done on the very data used to pick it. Only the held-out numbers speak about unseen data. Second, hypervolume values are comparable only between methods that share one normalisation, that is, within a row group of a table, and not between different scenarios.

6.2 Analysis

Verification was organised in three levels, from the smallest unit to the whole experiment.

- **Component level.** The repair operator is tested on random, all-zero, all-one and unit vectors, with and without a cardinality limit, and must always return weights that are non-negative, sum to one, do not exceed the cap and respect the limit. The objective function is checked against an independent calculation of mean, standard deviation and the CVaR average. The Pareto utilities are checked on a small hand-worked example.
- **Algorithm level.** Each optimiser must return only feasible, mutually non-dominated portfolios whose stored objective values match a fresh evaluation, must be reproducible for a fixed seed, and must beat random search at an equal budget.

- **System level.** The server is tested through a complete job, error paths and the HTTP interface, and the interface is exercised in a real browser.

The experiments add a further check on constraint handling: across all fronts produced in experiment E2, which includes limits of 10, 8 and 5 holdings, there were 0 violations of the 30% cap or of the holding limit.

6.3 Test results

6.3.1 Testing process

The automated tests are written with pytest and run in a few seconds. They are executed after each change. The 14 automated tests passed in the final run for this report. Interface tests were run in headless Chromium through Playwright against the running server, in a light desktop, a dark desktop and a phone-sized window.

6.3.2 System test cases

Table 6.1: System test cases and results

ID	Test case	Expected result	Result
T1	Repair on random, zero, ones and unit vectors, with no limit and with K = 5	Weights non-negative, sum to 1, each at most 30%, at most K non-zero	Pass
T2	Construct a problem with a 10% cap and a limit of 5 holdings	Error that the configuration is infeasible	Pass
T3	Objectives of the equal-weight portfolio versus a manual calculation	Return, volatility and CVaR agree	Pass
T4	Non-dominated mask, sorting ranks, crowding distance and hypervolume on a worked example	Expected mask and ranks; hypervolume grows with more points	Pass
T5	MOPSO and NSGA-II with a limit of 6 holdings	Feasible, mutually non-dominated; stored objectives match a re-evaluation; history length correct	Pass (both)
T6	Both algorithms versus random search at equal budget	Larger hypervolume for both algorithms	Pass
T7	Two runs of MOPSO with the same seed	Identical fronts	Pass
T8	Command-line run with walk-forward test	Result tables and figures written	Pass
T9	Complete job through the job manager	Status done; fronts, weights summing to 1, held-out statistics and picks present; JSON-serialisable	Pass
T10	Infeasible constraints through the job manager	Status error with an explanatory message	Pass
T11	Number of assets set to 500	Rejected with a message naming the field	Pass
T12	HTTP: home page, a parent-directory path, a run request and polling	Page served; traversal path returns 404; job completes	Pass

ID	Test case	Expected result	Result
T13	Browser: page load starts a run and results appear without console errors	Fronts drawn, inspector filled	Pass
T14	Browser: hover and click a point; switch algorithm, rule and axis	Tooltip appears; weights and metrics update; axes re-scale	Pass
T15	Browser: 10% cap with 5 holdings	Inline message appears and Run is disabled	Pass
T16	Browser: My prices without a file, then with a CSV of 8 assets	Run disabled without a file; run completes with the file’s tickers	Pass
T17	Browser: Stop during a large run	Run ends with the status Stopped	Pass
T18	Browser: phone width 390 px	No horizontal scrolling of the page (found and fixed during testing)	Pass
T19	Browser: dark colour scheme	All text and charts readable	Pass

Chapter 7

Results and Discussion

7.1 Presentation of Results

This section presents the measured results of the six experiments described in Section 4.2.1. Values after the plus-minus sign are sample standard deviations over seeds. In all experiments the algorithms received the same training data, the same constraints and the same number of objective evaluations.

E1: Front quality at the default setting

Table 7.1: Front quality over 10 seeds (20 assets, 100 particles or individuals, 150 iterations)

Method	Hypervolume	Front size	Spacing	Best Sharpe	Seconds	Evaluations
Particle swarm (MOPSO)	0.702 ± 0.027	100.0	0.0103	1.448	1.5	15,100
Genetic algorithm (NSGA-II)	0.706 ± 0.030	98.0	0.0160	1.447	1.3	15,100
Random search	0.254 ± 0.058	20.1	0.1271	0.987	<0.1	15,100

Both algorithms dominate random search by a wide margin (hypervolume about 0.70 against 0.25, Wilcoxon p = 0.002). Random search finds only about 20 non-dominated points, against about 100 for the optimisers. The mean hypervolume of NSGA-II is marginally higher than that of MOPSO (a difference of 0.0042, higher in 9 of 10 seeds, p = 0.014), but this is small compared with the seed-to-seed spread of about 0.027. MOPSO gives a more evenly spread front (spacing 0.0103 against 0.0160, p = 0.002). Run times were measured separately with nothing else running (see the scaling results below) and are about 1.5 s for MOPSO and 1.3 s for NSGA-II.

Selected portfolios on the default market

Table 7.2: Selected portfolios on the default market (seed 0): training and held-out performance

Portfolio	Held	Train return	Train vol.	Train Sharpe	Held-out CAGR	Held-out vol.	Held-out Sharpe	Held-out max DD
MOPSO best sharpe	4	22.7%	15.7%	1.448	-5.5%	13.6%	-0.350	-24.6%
MOPSO lowest volatility	12	10.4%	12.5%	0.828	-3.5%	11.2%	-0.261	-15.3%
MOPSO balanced	7	19.0%	14.1%	1.348	-4.9%	12.3%	-0.346	-19.7%
NSGA-II best sharpe	4	22.8%	15.7%	1.448	-5.6%	13.6%	-0.354	-24.6%
NSGA-II lowest volatility	12	9.0%	12.3%	0.727	-5.0%	11.1%	-0.406	-17.3%
NSGA-II balanced	6	19.0%	14.2%	1.342	-5.5%	12.3%	-0.397	-20.2%
Equal weight	20	5.6%	15.3%	0.362	-7.6%	13.5%	-0.521	-21.5%

On this one market all strategies lost money on the held-out window, including equal weight, because the held-out period is a weak stretch of the simulated path. The optimised portfolios lost less than equal weight (for example -5.5% a year against -7.6% for the best-Sharpe portfolio of MOPSO), but a single market cannot show whether this is skill or luck, which is why experiment E5 uses 30 markets. The best-Sharpe portfolios hold only four assets at the 30% cap, whereas the lowest-volatility portfolios hold twelve.

E2: Effect of the cardinality limit

Table 7.3: Effect of the holding limit (5 seeds each; 20 assets)

Max holdings K	Method	Hypervolume	Best Sharpe	Most holdings seen	Largest weight
None	MOPSO	0.700 ± 0.030	1.448	12	0.300
None	NSGA-II	0.703 ± 0.034	1.447	14	0.300
None	Random	0.263 ± 0.058	1.013	20	0.300
10	MOPSO	0.715 ± 0.028	1.448	10	0.300
10	NSGA-II	0.716 ± 0.032	1.447	10	0.300
10	Random	0.299 ± 0.052	1.060	10	0.300
8	MOPSO	0.731 ± 0.031	1.448	8	0.300
8	NSGA-II	0.732 ± 0.032	1.447	8	0.300
8	Random	0.330 ± 0.031	1.099	8	0.300
5	MOPSO	0.749 ± 0.026	1.448	5	0.300
5	NSGA-II	0.742 ± 0.029	1.447	5	0.300
5	Random	0.433 ± 0.045	1.292	5	0.300

The limit was respected in every run and the cap was never exceeded (0 violations). The best Sharpe ratio on the front is the same for every K because the best-Sharpe portfolio needs only four assets at a 30% cap, so even K = 5 does not bind there. The hypervolume of random search rises as K falls, since fewer holdings shrink the feasible region and make random sampling more effective, whereas the two optimisers stay close to each other. Hypervolumes are normalised within each K and must not be compared across K.

E3: Number of assets

Table 7.4: Scaling with the number of assets (3 seeds each for quality; run time measured separately with 3 seeds)

Assets	Method	Hypervolume	Seconds per run
10	MOPSO	0.677 ± 0.019	1.51
10	NSGA-II	0.682 ± 0.019	1.31
10	Random	0.546 ± 0.068	<0.1
20	MOPSO	0.581 ± 0.004	1.49
20	NSGA-II	0.581 ± 0.006	1.27
20	Random	0.424 ± 0.057	<0.1
40	MOPSO	0.672 ± 0.027	1.38
40	NSGA-II	0.668 ± 0.025	1.33
40	Random	0.232 ± 0.082	<0.1
60	MOPSO	0.617 ± 0.011	1.40
60	NSGA-II	0.590 ± 0.004	1.24
60	Random	0.124 ± 0.041	<0.1

Run time stays between about 1.2 and 1.5 seconds from 10 to 60 assets, because evaluation is a single matrix product whose cost is small next to the sorting and repair steps. The advantage over random search grows with dimension: with 60 assets the optimisers' hypervolume is about 0.62 and 0.59 against 0.12 for random search, whereas at 10 assets the gap is much smaller (0.68 and 0.68 against 0.55).

E4: Search budget

Table 7.5: Hypervolume against budget (5 seeds each; one fixed reference for all rows)

Iterations	Evaluations	MOPSO	NSGA-II	Random search
5	600	0.628 ± 0.040	0.369 ± 0.036	0.284 ± 0.051
10	1,100	0.752 ± 0.010	0.463 ± 0.049	0.330 ± 0.068
25	2,600	0.788 ± 0.003	0.646 ± 0.033	0.353 ± 0.075
50	5,100	0.797 ± 0.005	0.769 ± 0.010	0.381 ± 0.075
100	10,100	0.801 ± 0.002	0.804 ± 0.001	0.395 ± 0.076
200	20,100	0.802 ± 0.002	0.811 ± 0.002	0.430 ± 0.096

MOPSO is much better at small budgets: with 10 iterations (1,100 evaluations) it reaches a hypervolume of 0.75, which NSGA-II needs between 25 and 50 iterations to match. MOPSO was ahead of NSGA-II in all 5 seeds at budgets of 50 iterations or fewer. From 100 iterations on, NSGA-II was ahead in all 5 seeds, and at 200 iterations its hypervolume is 0.811 against 0.802. With five seeds the smallest attainable p value of the paired test is 0.062, which is what is reported for these unanimous comparisons. Random search improves slowly with budget and stays far below both optimisers.

E5: Held-out performance over 30 markets

Table 7.6: Held-out performance over 30 independent markets (mean over markets; p is the paired Wilcoxon test of held-out Sharpe against equal weight)

Strategy	Train Sharpe	Held-out Sharpe	Median	Held-out volatility	Held-out max DD	Wins vs EW	p vs EW
Equal weight	0.52	0.59 ± 0.76	0.50	14.7%	-16.2%	-	-
MOPSO best Sharpe	1.34	0.53 ± 0.81	0.56	16.6%	-19.0%	14/30	0.428
NSGA-II best Sharpe	1.34	0.53 ± 0.79	0.56	16.7%	-19.0%	13/30	0.416
Random search best Sharpe	1.06	0.51 ± 0.76	0.50	15.8%	-18.9%	11/30	0.221
MOPSO min volatility	0.64	0.61 ± 1.01	0.59	12.6%	-15.1%	12/30	0.871
NSGA-II min volatility	0.57	0.62 ± 1.00	0.59	12.4%	-14.6%	16/30	0.626
Random search min volatility	0.56	0.66 ± 0.89	0.58	13.2%	-14.9%	18/30	0.164

Best-Sharpe portfolios. The mean training Sharpe ratio of the chosen portfolios is 1.34 for MOPSO and 1.34 for NSGA-II, but on held-out data it falls to 0.53 and 0.53, which is not better than equal weight (0.59). The differences from equal weight are not statistically significant, and MOPSO and NSGA-II do not differ from each other ($p = 0.641$). Random search, which has a much poorer front, performs about as well out of sample.

Lowest-volatility portfolios. These behave differently. Their held-out volatility is lower than that of equal weight in 30 of 30 markets for MOPSO and 30 of 30 for NSGA-II (means 12.6% and 12.4% against 14.7%, $p < 0.001$). Their maximum drawdown is shallower than equal weight in 19 markets for MOPSO ($p = 0.061$) and 24 for NSGA-II ($p = 0.005$). Their held-out Sharpe ratio is similar to that of equal weight (0.61 and 0.62 against 0.59).

E6: Operator ablation

Table 7.7: Effect of removing an operator (8 seeds; limit of 8 holdings)

Variant	Hypervolume
PSO (full)	0.6607 ± 0.0131
PSO without turbulence	0.6589 ± 0.0131
GA (full)	0.6546 ± 0.0139
GA without swap mutation	0.6530 ± 0.0093

Removing the PSO turbulence step changed the hypervolume by 0.0017 on average ($p = 0.641$), and removing the GA swap mutation by 0.0016 ($p = 0.547$). Neither effect is distinguishable from seed-to-seed variation at this setting.

7.1.1 Statistical Analysis

Because the number of seeds is small and the distributions of hypervolume and Sharpe ratio are not known to be normal, paired Wilcoxon signed-rank tests were used instead of t-tests. All tests are two-sided and uncorrected for multiple comparisons, so they should be read as indicative, and p values near 0.05 should not be treated as firm.

Table 7.8: Summary of paired Wilcoxon signed-rank tests

Comparison	Pairs	Result	p
Hypervolume: MOPSO against random search	10	MOPSO higher in all pairs	0.002
Hypervolume: NSGA-II against random search	10	NSGA-II higher in all pairs	0.002
Hypervolume: MOPSO against NSGA-II	10	NSGA-II higher in 9 of 10; mean difference 0.0042	0.014
Spacing: MOPSO against NSGA-II	10	MOPSO more even	0.002
Hypervolume at 25 iterations: MOPSO against NSGA-II	5	MOPSO higher in all pairs (smallest attainable p)	0.062
Hypervolume at 200 iterations: MOPSO against NSGA-II	5	NSGA-II higher in all pairs (smallest attainable p)	0.062
Held-out Sharpe: MOPSO best Sharpe against equal weight	30	Not different	0.428
Held-out Sharpe: NSGA-II best Sharpe against equal weight	30	Not different	0.416
Held-out Sharpe: MOPSO against NSGA-II (best Sharpe)	30	Not different	0.641
Held-out volatility: MOPSO minimum volatility against equal weight	30	Lower in all 30	< 0.001
Held-out volatility: NSGA-II minimum volatility against equal weight	30	Lower in all 30	< 0.001
Held-out drawdown: NSGA-II minimum volatility against equal weight	30	Shallower in 24 of 30	0.005
Ablation: PSO with and without turbulence	8	Not different	0.641

Comparison	Pairs	Result	p
Ablation: GA with and without swap mutation	8	Not different	0.547

7.2 Figures

The figures in this section show single quantities. The first compares front quality at different budgets, the second contrasts the training and held-out Sharpe ratios, and the third shows the effect of removing an operator.

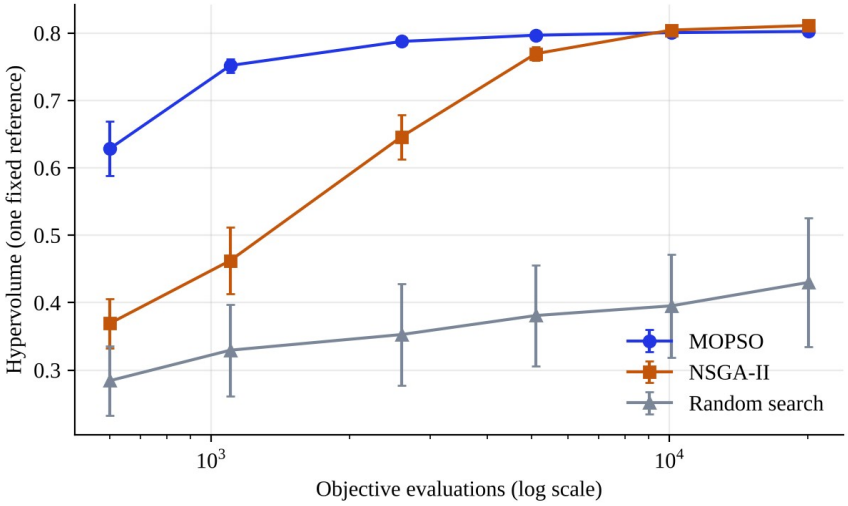


Figure 7.1: Hypervolume against the number of objective evaluations. Error bars are standard deviations over five seeds.

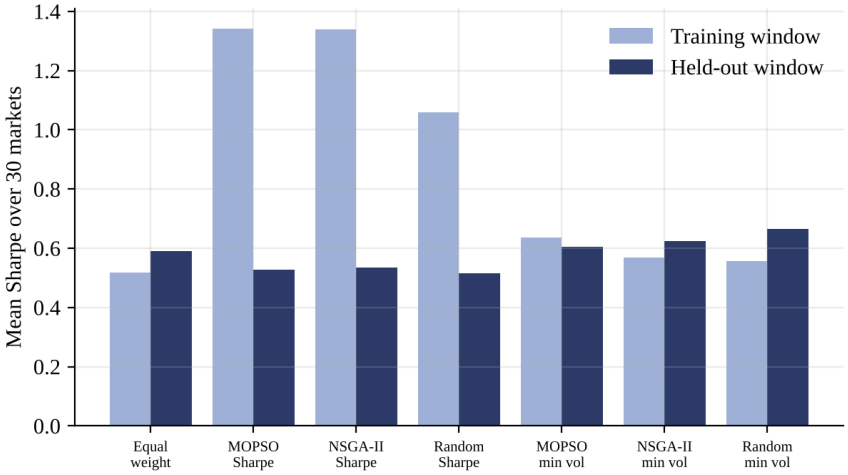


Figure 7.2: Mean Sharpe ratio on the training and the held-out window for each strategy, over 30 markets.

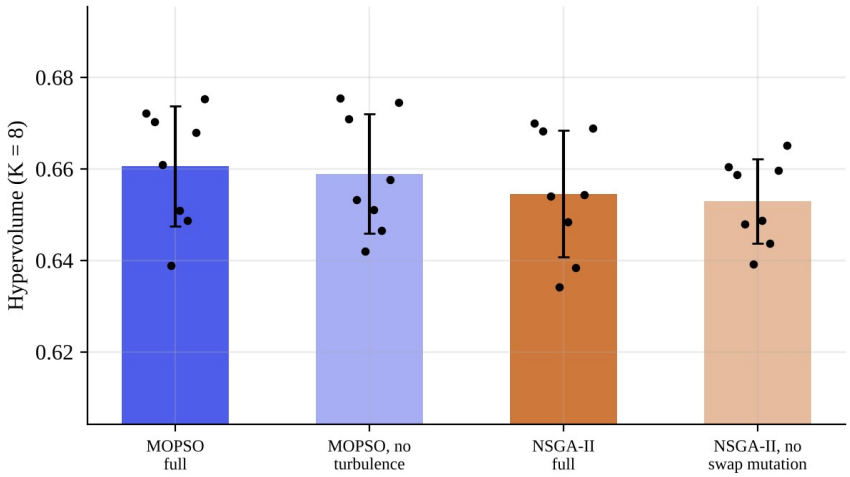


Figure 7.3: Hypervolume with and without the turbulence step (MOPSO) and the swap mutation (NSGA-II). Dots are individual seeds.



Figure 7.4: Interface: held-out wealth curve of the selected portfolio against equal weight, with the comparison scorecard and the convergence chart.

7.3 Subfigures

The multi-panel figures below present the fronts, the convergence and the distributions behind the tables. Panels are labelled (a), (b) and (c) in the figures.

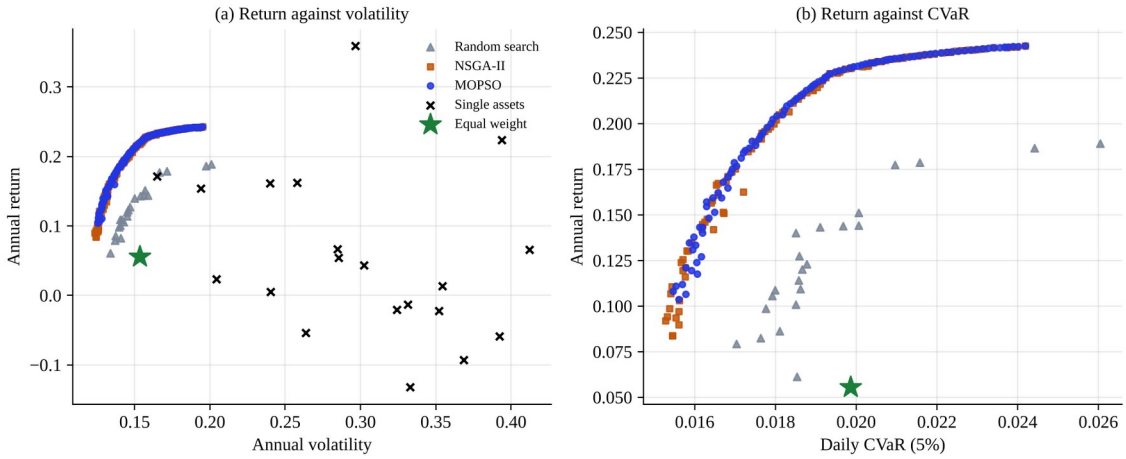


Figure 7.5: Fronts found on the training window of the default market: (a) return against volatility; (b) return against CVaR.

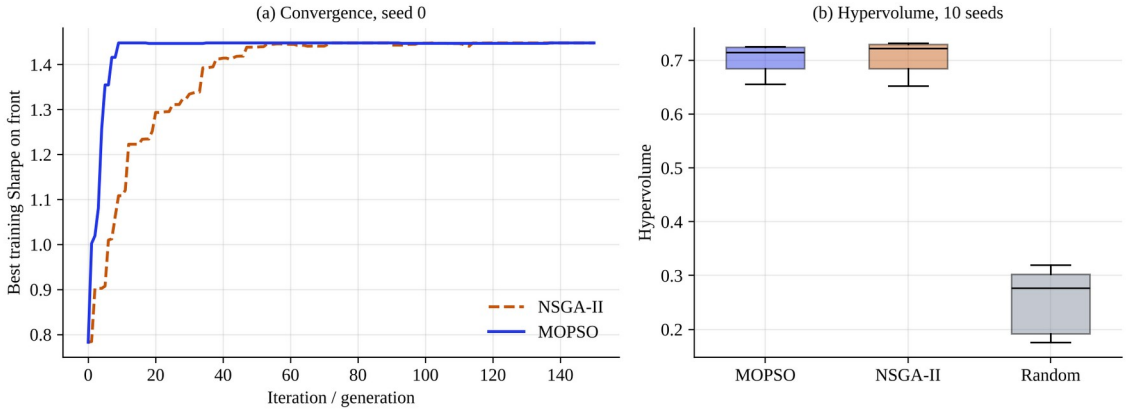


Figure 7.6: Best training Sharpe on the front per iteration for seed 0 (a) and hypervolume distribution over ten seeds (b).

In Figure 7.6(a), MOPSO reaches 99% of its final best Sharpe ratio after 9 iterations and NSGA-II after 47 iterations, and both end at the same value (1.448).

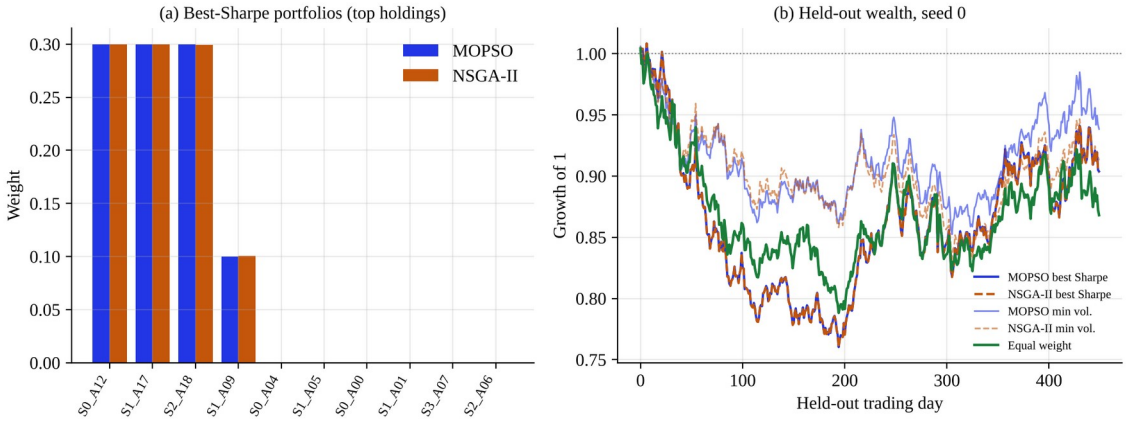


Figure 7.7: Best-Sharpe portfolios on the default market: (a) weights of the largest holdings; (b) held-out wealth of selected portfolios against equal weight.

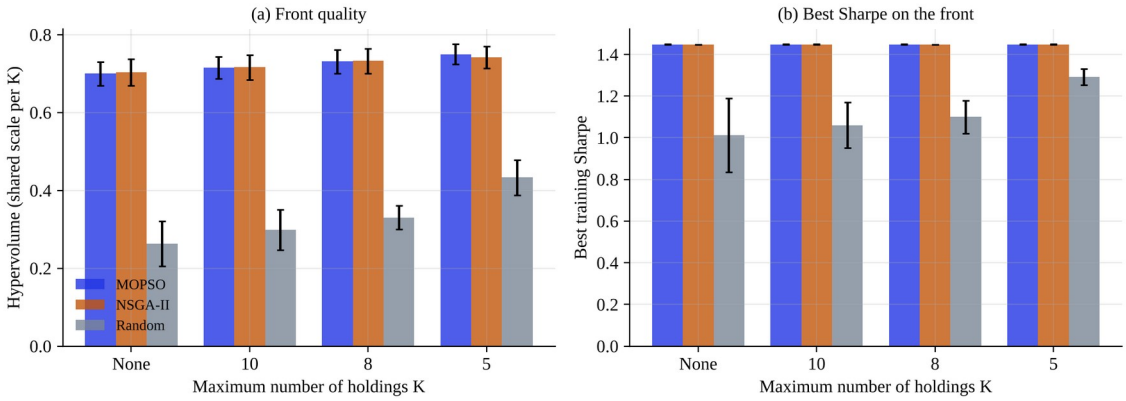


Figure 7.8: Effect of the holding limit K: (a) hypervolume; (b) best training Sharpe on the front.

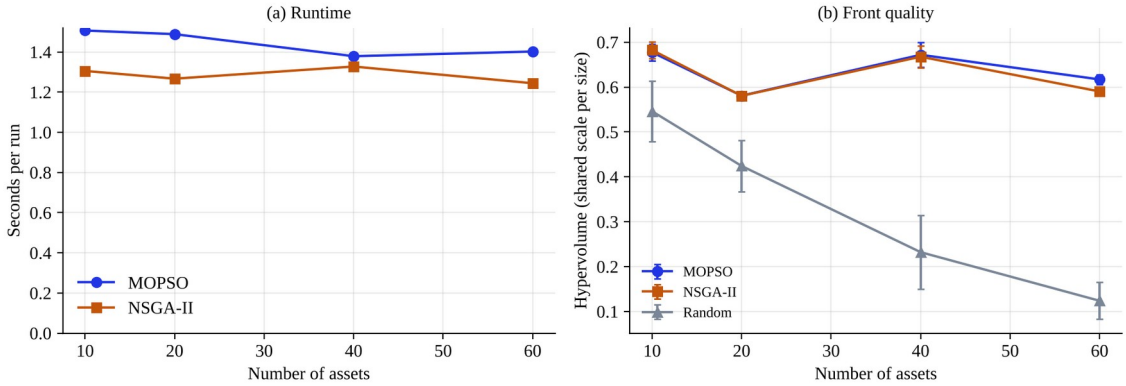


Figure 7.9: Scaling with the number of assets: (a) measured run time; (b) hypervolume.

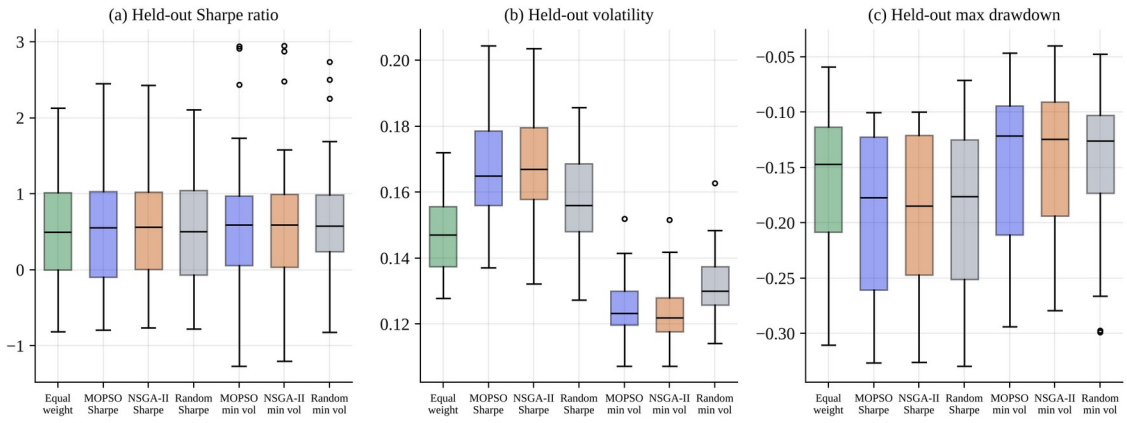


Figure 7.10: Held-out distributions over 30 markets: (a) Sharpe ratio; (b) volatility; (c) maximum drawdown. Boxes show quartiles and whiskers the range excluding outliers.

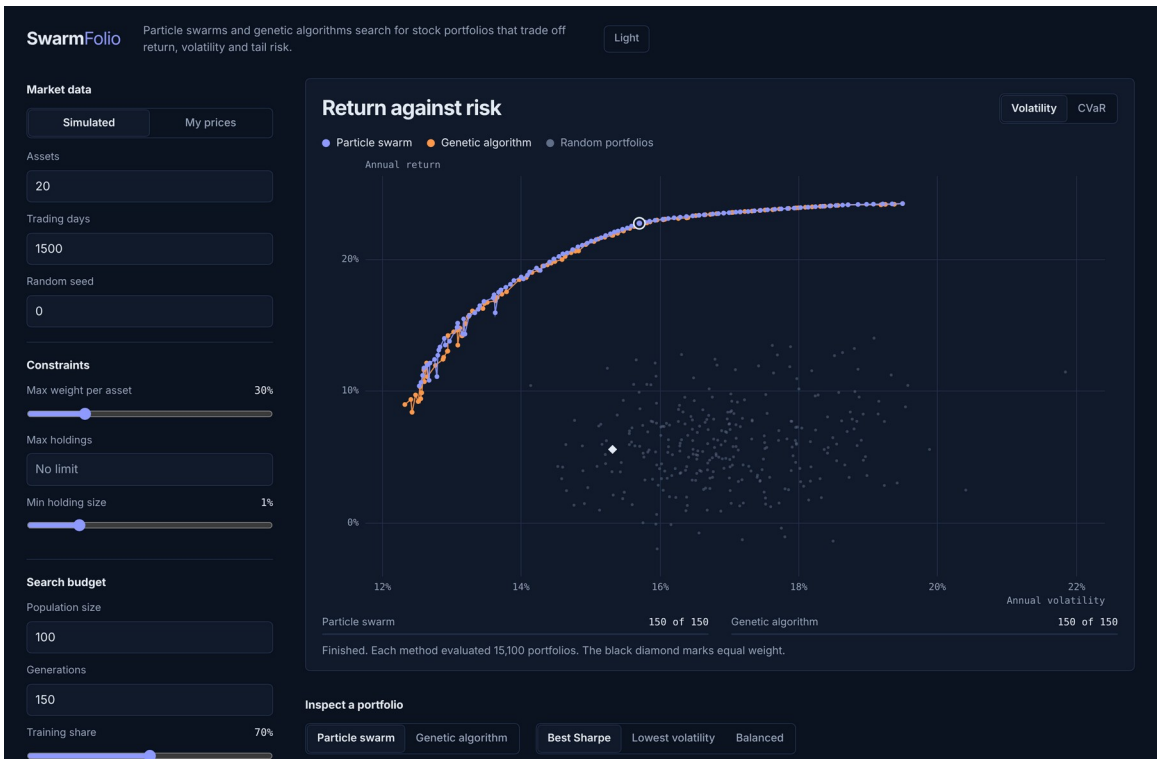


Figure 7.11: The same screen in the dark colour scheme.

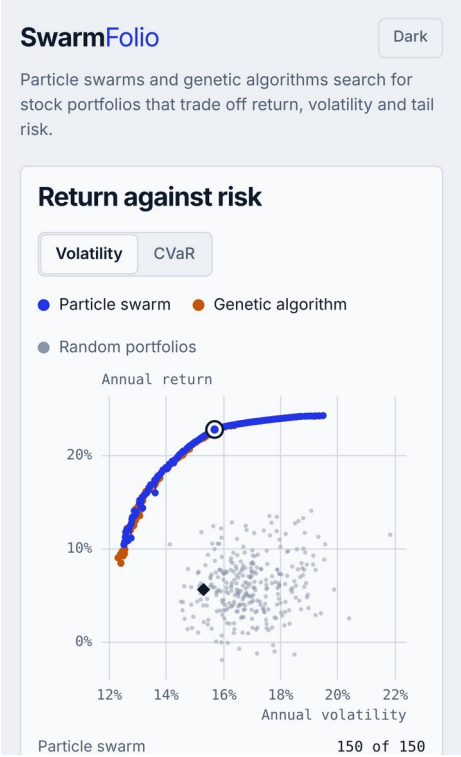


Figure 7.12: Phone-width layout: the stage comes first and the controls follow.

7.4 Discussion

Both optimisers approximate the front well and are close to each other. Both reach a hypervolume of about 0.70 against about 0.25 for random search at the same budget, and they return about 100 non-dominated portfolios each. NSGA-II is marginally ahead on hypervolume at the default budget and MOPSO has the more even front, but the differences

are small. The budget study explains why the ranking depends on how the comparison is set up: MOPSO converges in far fewer iterations (it reaches a hypervolume of 0.75 after 10 iterations, a level that NSGA-II reaches after between 25 and 50), so it wins at small budgets, whereas NSGA-II keeps improving and finishes slightly ahead when given 100 iterations or more. A fair statement is that neither algorithm is better overall, MOPSO is preferable when evaluations are scarce, and NSGA-II is preferable when they are plentiful. This is consistent with a comparison at one budget being able to favour either algorithm.

Constraints are handled reliably and do not penalise the optimisers. Using one repair operator for both algorithms meant that no run in the constraint study violated the cap or the holding limit. Tight limits do not hurt the best Sharpe ratio in this market because the best portfolio holds only four assets, while random search profits from the smaller feasible region.

The cost of a run is small and almost independent of the number of assets. Evaluation is a single matrix product for the whole population, so a default run takes about a second and a half on four cores. The advantage over random search widens as the universe grows, as expected for a search space that grows with the number of assets.

Operator ablations show no measurable benefit. The turbulence step and the swap mutation were added to keep the populations diverse, but removing them changed the hypervolume by less than 0.002 on average, well within seed-to-seed variation. The honest conclusion is that at these settings they are harmless safeguards and not drivers of performance. It remains possible that they matter in harder or longer runs.

A better front does not mean a better portfolio on new data. The most important result for a practitioner is in E5. Portfolios chosen for the best Sharpe ratio looked excellent in training (1.34) and fell to 0.53 on unseen data, below the equal-weight portfolio at 0.59. Random search, with a far worse front, did about as well. This agrees with the arguments of Michaud [7] and the evidence of DeMiguel et al. [8]. A simple calculation shows why it is expected in these markets: with about four years of data, the standard error of an annual mean return is roughly the asset's annual volatility divided by the square root of the number of years, which for an asset with 25% volatility is about 12 percentage points. That is as large as the whole spread of true alphas used to generate the markets (from -2% to 10% a year), so the optimiser mostly fits noise in the means.

Risk-based choices do transfer. In contrast with returns, volatility is estimated far more precisely from the same data. The lowest-volatility portfolios had lower held-out volatility than equal weight in all 30 markets and often shallower drawdowns, while earning a similar

Sharpe ratio. In the practical use of the tool, selecting from the low-risk end of the front is therefore the more dependable choice, and the high-return end should be treated sceptically. Because random search also finds acceptable minimum-volatility portfolios, the main value of the metaheuristics lies in producing a full, well-spread front rather than in a single extreme point.

Limitations

- All markets are simulated. Real prices have time-varying correlations, trends and regime changes that the generator does not capture, so the held-out findings need to be repeated on real data before any practical conclusion is drawn.
- Hypervolume is estimated by sampling and depends on the reference point. Only comparisons within a shared normalisation are made.
- The algorithm parameters (inertia, acceleration constants, distribution indices) were fixed at common values and not tuned, and a different setting could change the ranking.
- Two optimisers, one risk-free rate of zero, long-only portfolios and no trading costs in the main experiments.
- The tests are uncorrected for multiple comparisons, and five to ten seeds give limited power.

7.5 Conclusion

The experiments show that both MOPSO and NSGA-II find much better fronts than random search under identical constraints and budget, that they are close to one another with MOPSO faster at small budgets and NSGA-II marginally better at large ones, and that portfolios chosen for return do not beat equal weight on unseen data while low-risk portfolios keep their risk advantage. The next chapter summarises the conclusions and future work.

Chapter 8

Conclusion

8.1 Conclusion

This project built SwarmFolio, a library and a browser workbench for multi-objective portfolio selection with a particle swarm optimiser and a genetic algorithm. Both optimisers search one shared constrained space of long-only portfolios, trading off annual return, volatility and CVaR, and both are checked by an automated test suite and by an interface test in a real browser.

The main findings are as follows.

- Both algorithms approximate the Pareto front far better than random search at an equal budget (hypervolume about 0.70 against 0.25), and respect every constraint in every run.
- MOPSO converges in far fewer iterations and gives a more even front. NSGA-II ends marginally ahead when given 100 or more iterations. The two cannot be separated in practical terms at the default budget.
- A run takes about a second and a half and the cost is nearly independent of the number of assets up to 60.
- The PSO turbulence step and the GA swap mutation did not measurably change front quality at the tested settings.
- Portfolios chosen for the best Sharpe ratio on training data did not outperform the equal-weight portfolio on held-out data in 30 simulated markets, while minimum-volatility portfolios kept clearly lower held-out volatility in all 30. Optimisation quality on the training window should therefore not be read as a promise of future return.
- The interface makes the search observable and each portfolio inspectable, and lets users bring their own prices.

The study has limits that are stated in Section 7.4: the markets are simulated, parameters were not tuned and the number of seeds is modest. The conclusions about out-of-sample behaviour are consistent with the literature, but are demonstrated here only on simulated data.

8.2 Future Scope

- **Real-market validation.** Repeat the held-out study on historical prices of a stock index across many rolling windows, using the CSV import that already exists.
- **Better risk and return estimation.** Add shrinkage covariance estimators and robust return estimates, which are the standard remedy for the estimation-error problem seen in the held-out results.
- **More objectives and constraints.** Add turnover and transaction cost as objectives, sector limits, and a minimum-lot constraint.
- **More algorithms.** Add decomposition-based and indicator-based evolutionary algorithms, and hybrid swarm-genetic variants, and tune parameters with a systematic search.
- **Exact hypervolume and statistical rigour.** Use an exact hypervolume routine and correct for multiple comparisons, with more seeds.
- **Deployment.** Add authentication, rate limiting and an isolated worker process so that the workbench can be hosted for a class, and store jobs and results on disk.
- **Education features.** Add guided exercises in the interface that explain dominance, the effect of constraints and over-fitting on the user’s own run.

B.M.S. College of Engineering.

Department of AI & DS

Attainment of POs and PSOs

Batch no.: _____ Date: _____

Project Title: SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms

PROGRAM OUTCOMES

PO	Level (3/2/1) 3-High 2-Medium 1-Low	Justification if addressed
PO1	3	Portfolio theory, probability and statistics, and multi-objective optimisation were applied to formulate and solve the problem in software.
PO2	3	The problem was formulated with three objectives and four constraints; literature was reviewed (Chapter 2), gaps identified and conclusions supported by experiments.
PO3	2	A layered system (library, server, interface) was designed and built to meet the requirements in Chapter 3; public-health and environmental considerations are not directly relevant.
PO4	3	Six controlled experiments with seeds, statistical tests and ablations were designed and analysed (Chapters 4 and 7).
PO5	3	NumPy, pandas, Matplotlib, SciPy, pytest and a browser canvas were used, and their limitations (for example the Monte Carlo hypervolume) are stated.
PO6	1	Societal relevance is discussed through the SDGs (Section 1.5); sustainability impact is limited.
PO7	2	Sources are cited in IEEE style, results are reported without selection, and the report states that the tool is not investment advice.
PO8	3	The work was divided among three team members across modules, experiments and documentation, and integrated through one repository.
PO9	2	A structured report, figures and an interactive interface communicate the work to technical and non-technical readers.
PO10	2	A work plan, cost estimate and risk register were prepared and followed (Sections 1.8, 3.5 and 3.7).
PO11	2	The team learned multi-objective metaheuristics, statistical evaluation and browser-based visualisation, which were new to the course content.

PROGRAM SPECIFIC OUTCOMES

PSO	Level (3/2/1) 3-High 2-Medium 1-Low	Justification if addressed
PSO1	3	A computing system using AI techniques (swarm and evolutionary search) was designed and developed.
PSO2	2	Optimisation and statistical methods were applied to a financial decision problem and its results were interpreted for decision making.
PSO3	3	An end-to-end system was built, from data generation and import through optimisation and evaluation to an interactive interface and tests.

REFERENCES

[1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77-91, Mar. 1952.

[2] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *The Journal of Risk*, vol. 2, no. 3, pp. 21-41, 2000.

[3] T.-J. Chang, N. Meade, J. E. Beasley, and Y. M. Sharaiha, "Heuristics for cardinality constrained portfolio optimisation," *Computers & Operations Research*, vol. 27, no. 13, pp. 1271-1302, 2000.

[4] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks*, vol. 4, Perth, WA, Australia, 1995, pp. 1942-1948.

[5] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. Michigan Press, 1975.

[6] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182-197, Apr. 2002.

[7] R. O. Michaud, "The Markowitz optimization enigma: Is 'optimized' optimal?" *Financial Analysts Journal*, vol. 45, no. 1, pp. 31-42, Jan.-Feb. 1989.

[8] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?" *The Review of Financial Studies*, vol. 22, no. 5, pp. 1915-1953, May 2009.

[9] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015. [Online]. Available: https://sdgs.un.org/2030agenda. [Accessed: Oct. 30, 2026].

[10] T. Cura, "Particle swarm optimization approach to portfolio optimization," *Nonlinear Analysis: Real World Applications*, vol. 10, no. 4, pp. 2396-2406, Aug. 2009.

[11] H. Zhu, Y. Wang, K. Wang, and Y. Chen, "Particle swarm optimization (PSO) for the constrained portfolio optimization problem," *Expert Systems with Applications*, vol. 38, no. 8, pp. 10161-10169, Aug. 2011.

[12] G.-F. Deng, W.-T. Lin, and C.-C. Lo, "Markowitz-based portfolio selection with cardinality constraints using improved particle swarm optimization," *Expert Systems with Applications*, vol. 39, no. 4, pp. 4558-4566, Mar. 2012.

[13] K. P. Anagnostopoulos and G. Mamanis, "A portfolio optimization model with three objectives and discrete variables," *Computers & Operations Research*, vol. 37, no. 7, pp. 1285-1297, Jul. 2010.

[14] P. Skolpadungket, K. Dahal, and N. Harnpornchai, "Portfolio optimization using multi-objective genetic algorithms," in *Proc. IEEE Congr. Evolutionary Computation*, Singapore, 2007, pp. 516-523.

[15] K. Metaxiotis and K. Liagkouras, "Multiobjective evolutionary algorithms for portfolio management: A comprehensive literature review," *Expert Systems with Applications*, vol. 39, no. 14, pp. 11685-11698, Oct. 2012.

[16] W. F. Sharpe, "Mutual fund performance," *The Journal of Business*, vol. 39, no. 1, pt. 2, pp. 119-138, Jan. 1966.

[17] K. Deb and R. B. Agrawal, "Simulated binary crossover for continuous search space," *Complex Systems*, vol. 9, no. 2, pp. 115-148, 1995.

[18] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach," *IEEE Trans. Evol. Comput.*, vol. 3, no. 4, pp. 257-271, Nov. 1999.

[19] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE Int. Conf. Evolutionary Computation*, Anchorage, AK, USA, 1998, pp. 69-73.

[20] C. A. Coello Coello, G. T. Pulido, and M. S. Lechuga, "Handling multiple objectives with particle swarm optimization," *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 256-279, Jun. 2004.

[21] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. Grunert da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Trans. Evol. Comput.*, vol. 7, no. 2, pp. 117-132, Apr. 2003.

[22] J. R. Schott, "Fault tolerant design using single and multicriteria genetic algorithm optimization," M.S. thesis, Dept. Aeronaut. Astronaut., Massachusetts Inst. Technol., Cambridge, MA, USA, 1995.

[23] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester, U.K.: Wiley, 2001.

[24] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, Sep. 2020.

[25] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, Austin, TX, USA, 2010, pp. 56-61.

[26] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, May-Jun. 2007.

[27] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261-272, Mar. 2020.

[28] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80-83, Dec. 1945.

APPENDIX A: Details of list of publications related to this project

No publication related to this project is listed at the time of writing. If a paper is submitted or accepted, add its details here, together with the certificate of publication and the published paper.

APPENDIX B: POs and PSOs Mapped

The Program Outcomes and Programme Specific Outcomes of the Department of AI & DS, BMSCE, are reproduced below with the project components that address them. The levels and justifications are given on the attainment sheet at the end of Chapter 8.

Table 8.1: Program Outcomes (PO) mapped to the project

PO	Statement	Addressed in
PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.	Chapters 1 to 5: portfolio model, objectives, algorithms
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)	Chapter 2 and Chapter 4: literature, gaps and problem formulation
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)	Chapters 3 to 5: requirements, architecture and implementation
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).	Chapters 4, 6 and 7: experimental design, statistical analysis
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)	Chapter 5 and Section 3.4: NumPy, pandas, SciPy, Matplotlib, pytest
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).	Section 1.5: SDGs
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)	Sections 3.7.1 and 5.4; reference list
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.	Team organisation of the project
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences	This report and the interactive interface
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.	Sections 1.8, 3.5 and 3.7
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)	Whole project

Table 8.2: Programme Specific Outcomes (PSO) mapped to the project

PSO	Statement	Addressed in
PSO1	Design and develop computing systems using principles and concepts of Artificial Intelligence and Data science	Chapters 4 and 5: design and implementation
PSO2	Apply the concepts, principles of functional data science, machine learning and AI tools and techniques to address common challenges and make better-informed decisions in business processes	Chapter 7: results interpreted for investment decisions
PSO3	Effectively build end-to-end Data Science, & AI systems/projects and solve complex interdisciplinary problems	Whole system: data, optimisation, evaluation, interface, tests

APPENDIX C: AI Generated Report and Plagiarism report Screen shot

Insert here the screenshots of the AI-generated-content report and the plagiarism (similarity) report produced by the institution’s checking tool for this report, as required by the department. These reports can only be generated from the final document after it is submitted to the tool, so this page is left for the team to complete.